CS 150: FUNDAMENTALS OF COMPUTER SCIENCE

Overview

Computer science is the study of computation, information, and automation. Included broadly in the sciences, computer science spans theoretical disciplines (such as algorithms, theory of computation, and information theory) to applied disciplines (including the design and implementation of hardware and software). An expert in the field is known as a computer scientist.

Algorithms and data structures are central to computer science. The theory of computation concerns abstract models of computation and general classes of problems that can be solved using them. The fields of cryptography and computer security involve studying the means for secure communication and preventing security vulnerabilities. Computer graphics and computational geometry address the generation of images. Programming language theory considers different ways to describe computational processes, and database theory concerns the management of repositories of data. Human–computer interaction investigates the interfaces through which humans and computers interact, and software engineering focuses on the design and principles behind developing software. Areas such as operating systems, networks and embedded systems investigate the principles and design behind complex systems. Computer architecture describes the construction of computer components and computer-operated equipment. Artificial intelligence and machine learning aim to synthesize goal-orientated processes such as problem-solving, decision-making, environmental adaptation, planning and learning found in humans and animals. Within artificial intelligence, computer vision aims to understand and process image and video data, while natural language processing aims to understand and process textual and linguistic data.

The fundamental concern of computer science is determining what can and cannot be automated.

COURSE OUTLINE BY ELISHA

1. Algorithm Design and Problem Solving
2. Programming
3. Databases

VOLUME 1

CHAPTER 7: ALGORITHM DESIGN AND PROBLEM SOLVING

PART 7.1: WHAT IS AN ALGORITHM

7.1.1 Everyday Meaning of "Algorithm"

Before we talk about computers, let's understand what an algorithm is in everyday life.

Simple definition: An algorithm is a **step-by-step procedure** to solve a problem or accomplish a task.

Example 1 – Making a cup of tea:

Step 1: Fill kettle with water

Step 2: Boil the water

Step 3: Put a tea bag in a cup

Step 4: Pour hot water into the cup

Step 5: Wait 3 minutes

Step 6: Remove the tea bag

Step 7: Add milk and sugar (if desired)

Step 8: Stir

Step 9: Drink

Example 2 – Brushing your teeth:

Step 1: Pick up toothbrush

Step 2: Apply toothpaste

Step 3: Turn on tap

Step 4: Wet toothbrush

Step 5: Brush top teeth for 30 seconds

Step 6: Brush bottom teeth for 30 seconds

Step 7: Rinse mouth

Step 8: Clean toothbrush

Step 9: Put toothbrush away

Key observation: Every algorithm has:

1. **A starting point** (step 1)
2. **Clear instructions** (each step is unambiguous)
3. **An ending point** (final step)

7.1.2 Computer Algorithm Definition

A **computer algorithm** is the same idea, but the steps are written in a way that a computer can understand and execute.

Important: Computers are extremely literal. They do exactly what you tell them, not what you *meant*.

Example of bad algorithm for a computer:

"Make a sandwich"

A computer cannot do this because it's too vague. What kind of bread? What filling? How to spread?

Correct computer algorithm for a sandwich:

Step 1: Take two slices of bread

Step 2: Open the jar of peanut butter

Step 3: Insert knife into jar

Step 4: Scoop out 10 grams of peanut butter

Step 5: Spread peanut butter evenly on one slice of bread

Step 6: Close the jar

Step 7: Place second slice on top of the first

Step 8: Serve

7.1.3 Why Do We Need Algorithms?

Three reasons:

1. **Precision** – Computers cannot guess. Algorithms remove ambiguity.
2. **Efficiency** – A good algorithm solves a problem faster than a bad one.
3. **Reusability** – Once written, an algorithm can solve many similar problems.

Real-world example – Finding a name in a phone book:

- **Bad algorithm:** Read every name from page 1 until you find it (slow).
- **Good algorithm:** Open to the middle, see if your name comes before or after, discard half the book, repeat (fast).

Both work. The second is better. That's why algorithm design matters.

7.1.4 Characteristics of a Good Algorithm

A good algorithm must have:

Characteristic	Meaning	Example (Bad vs Good)
Finiteness	Must end after a finite number of steps	"Count all numbers" (never ends) vs "Count from 1 t 100" (ends)
Definiteness	Each step is exactly clear	"Add a little salt" vs "Add 2 grams of salt"
Input	Zero or more inputs	"Find the average" (needs numbers to average)
Output	At least one result	"Add two numbers" (outputs the sum)
Effectiveness	Each step can be done exactly	"Calculate square root of -1" (impossible with real numbers)

7.1.5 How Are Algorithms Represented?

Three main ways:

1. **Plain English** (as above) – easy for humans, too vague for computers
2. **Flowchart** – visual diagram with shapes
3. **Pseudocode** – halfway between English and programming language

---

PART 7.2: PROGRAM DEVELOPMENT LIFE CYCLE (PDLC)

7.2.1 What is the PDLC?

The **Program Development Life Cycle** is a structured process that software developers follow to create high-quality programs. Think of it like a recipe for building software.

Why follow a process?

- Without a process: chaos, missed requirements, bugs, late delivery
- With a process: organized, testable, maintainable, on time

7.2.2 The Six Phases

PHASE 1: ANALYSIS → What is the problem?

PHASE 2: DESIGN → How will we solve it?

PHASE 3: CODING → Write the program

PHASE 4: TESTING → Find and fix errors

PHASE 5: MAINTENANCE → Update after release

PHASE 6: EVALUATION → Did it work?

Let's explore each phase in extreme detail.

PHASE 1: ANALYSIS (Deep Dive)

Goal: Understand exactly what the user needs.

Key question: What is the problem we are trying to solve?

Step 1.1: Gather Requirements

Methods to gather requirements:

Method	How it works	When to use
Interviews	One-on-one conversations with users	Small systems, detailed needs
Questionnaires	Written questions sent to many users	Large systems, statistical data
Observation	Watch users doing their current job	When users can't explain what they d
Document analysis	Study existing forms, reports, manuals	When converting paper to computer

Example – Building a library system:

Interview with librarian:

- *"What do you need the system to do?"* → "Track which books are borrowed"

- *"What information about a book?"* → "Title, author, ISBN, shelf location"
- *"What happens when a book is overdue?"* → "Send a reminder email"
- *"How many users?"* → "5000 students, 200 staff"

Step 1.2: Identify Inputs, Processes, Outputs

Every program has:

- **Input** – data that goes in (user typing, file, sensor)
- **Process** – calculations or decisions (adding numbers, checking conditions)
- **Output** – results displayed or saved (screen, file, printer)

Example – Grading system:

INPUTS:

- Student name (text)

- Marks for each subject (numbers 0-100)

PROCESSES:

- Calculate total marks

- Calculate percentage

- Determine grade using rules:

  70+ = A

  60-69 = B

  50-59 = C

  40-49 = D

  Below 40 = Fail

OUTPUTS:

- Student name

- Percentage

- Grade

- "Pass" or "Fail" message

Step 1.3: Identify Constraints

Constraints are limitations on the solution.

Types of constraints:

- **Budget** – How much money can be spent?
- **Time** – Deadline for delivery
- **Technology** – Must run on existing hardware
- **Legal** – Data protection laws (GDPR)
- **Skills** – Team knows certain programming languages

Example: "The grading system must run on the school's Windows 10 computers, be completed in 2 weeks, and cost nothing beyond developer salary."

Step 1.4: Write Requirements Specification

This is a formal document that becomes the contract between client and developer.

Template:

PROJECT NAME: Student Grading System

DATE: [today's date]

AUTHOR: [your name]

1. INTRODUCTION

   Purpose: Automate calculation of student grades

2. FUNCTIONAL REQUIREMENTS (what it must do)

   2.1 Accept marks for up to 10 subjects per student

   2.2 Calculate percentage (total/possible*100)

   2.3 Assign letter grade based on table

   2.4 Display results on screen

   2.5 Save results to a file

3. NON-FUNCTIONAL REQUIREMENTS (quality attributes)

   3.1 Response time under 1 second

3.2 Handle 1000 students simultaneously

3.3 99.9% uptime

3.4 User-friendly interface

4. CONSTRAINTS

4.1 Must run on Windows 10

4.2 Budget: $0 (existing hardware/software)

4.3 Deadline: 14 days

5. ACCEPTANCE CRITERIA (how to know it's done)

5.1 All functional requirements work

5.2 No crashes during normal use

5.3 User testing with 5 teachers shows no confusion

Step 1.5: Feasibility Study

Before starting, check if the project is even possible.

Four types of feasibility:

Type	Question
Technical	Do we have the technology and skills?
Economic	Can we afford it? Will it save money?
Legal	Are there any laws preventing this?
Operational	Will users actually use it?

Example – Library system:

- Technical: Yes (we know Python and SQL)
- Economic: Yes (savings from manual labor > development cost)

- Legal: Yes (data stored on local server, complies with privacy laws)
- Operational: Yes (librarians want this)

Decision: Proceed to design phase.

PHASE 2: DESIGN

Goal: Create a detailed plan that a programmer can follow.

Key question: How exactly will the software work?

7.2.3 Top-Down Design (Stepwise Refinement)

What is top-down design?
Breaking a big problem into smaller, manageable sub-problems. Then breaking those sub-problems even further. Continue until each piece is simple enough to code easily.

Analogy – Writing a report:

Write Report

├── Research topic

│ ├── Find sources online

│ ├── Read sources

│ └── Take notes

├── Create outline

│ ├── Introduction

│ ├── Body paragraphs

│ └── Conclusion

├── Write draft

│ ├── Write introduction

│ ├── Write each body paragraph

│ └── Write conclusion

├── Revise

│ ├── Check for clarity

│ └── Fix grammar

└── Submit

Example – Payroll system:

Calculate Payroll

├── Get employee data

│ ├── Input employee ID

│ ├── Input hours worked

│ └── Input hourly rate

├── Calculate gross pay

│ ├── If hours ≤ 40: gross = hours × rate

│ └── If hours > 40: gross = 40×rate + (hours-40)×rate×1.5

├── Calculate deductions

│ ├── Tax = gross × 0.2

│ ├── Social security = gross × 0.05

│ └── Health insurance = 50 (fixed)

├── Calculate net pay

│ └── net = gross - (tax + social + insurance)

└── Print payslip

├── Print employee name

├── Print gross pay

├── Print each deduction

└── Print net pay

Benefits of top-down design:

1. **Manages complexity** – You never think about the whole huge problem at once
2. **Enables teamwork** – Different programmers work on different modules
3. **Makes testing easier** – Test each module separately
4. **Promotes reuse** – Modules can be used in other programs

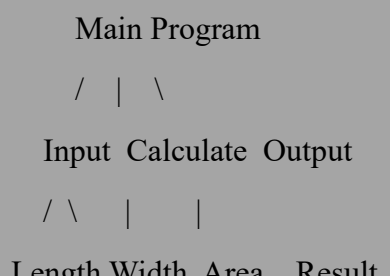
7.2.4 Structure Diagrams (Hierarchy Charts)

A **structure diagram** visually shows the modules and their relationships.

Rules for structure diagrams:

- Main module at the top (root)
- Sub-modules below
- Lines connect modules to their sub-modules
- No cycles (cannot go back up)

Example – Calculate Rectangle Area:



Drawing symbols:

- Rectangle = module (a subroutine/procedure)
- Line = "calls" or "uses"
- No arrows needed (direction is top-to-bottom)

When to use structure diagrams:

- Planning before coding
- Explaining design to non-programmers
- Documentation for future maintenance

7.2.5 Flowcharts (Detailed)

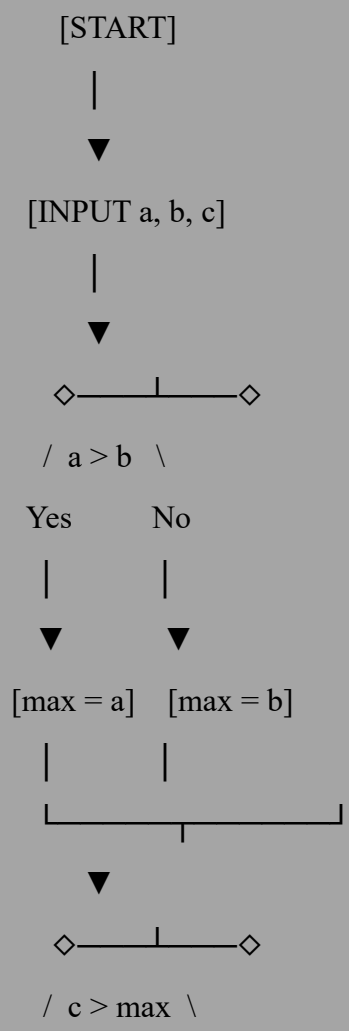
A **flowchart** shows the sequence of steps in an algorithm using standard symbols.

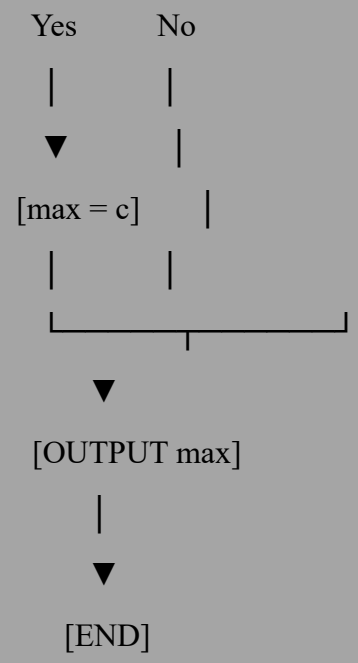
The Standard Flowchart Symbols (MEMORIZE THESE):

Symbol	Name	Meaning	Example
◯	Terminator	Start or end of program	START, END
▭	Process	An action or calculation	x = x + 1
▱	Input/Output	Getting data or displaying results	INPUT name

Symbol	Name	Meaning	Example
◇	Decision	Yes/no question	IF x>5?
⟳	Loop	Repeat a section	FOR i=1 TO 10
→	Flow line	Direction of sequence	connects shapes
	Document	Print or display document	PRINT report
	Database	Database operation	SAVE to DB

Detailed example – Find largest of three numbers:



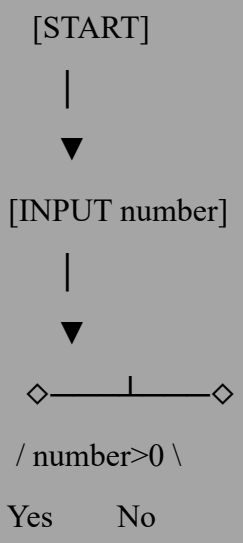


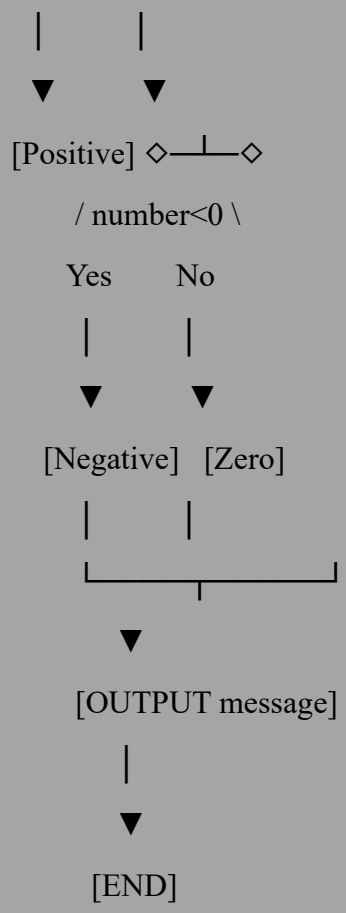
How to read a flowchart:

1. Start at the top (START)
2. Follow arrows
3. At diamonds, choose the path that matches the condition
4. End at END

Practice exercise – Draw a flowchart for:
"Input a number. If it's positive, output 'Positive'. If negative, output 'Negative'. If zero, output 'Zero'."

Answer:





7.2.6 Pseudocode (Introduction)

What is pseudocode?
A cross between English and programming language.

It is not real code (a computer cannot run it), but it is structured enough that a programmer can easily translate it to real code.

Why use pseudocode?

- Faster than flowcharts for complex algorithms
- Language-independent (works for Python, Java, C++, etc.)
- Easier to modify than diagrams
- Standard in computer science exams (IGCSE, A-Level, IB)

Basic pseudocode rules (Cambridge / IGCSE standard):

Construct	Pseudocode Syntax
Assignment	x ← 5 or x = 5
Input	INPUT variable
Output	OUTPUT "text" or OUTPUT variable

IF statement	IF condition THEN
	statements
	ELSE
	statements
	ENDIF

FOR loop	FOR i ← 1 TO 10 statements NEXT i

WHILE loop	WHILE condition DO
	statements
	ENDWHILE

REPEAT UNTIL	REPEAT statements UNTIL condition
Procedure	PROCEDURE name(parameters) statements ENDPROCEDURE
Function	FUNCTION name(parameters) RETURNS type statements RETURN value ENDFUNCTION
Array	DECLARE name[size] : DATATYPE

Example 1 – Simple sequence:

```
INPUT studentName
INPUT testScore
OUTPUT "Student: ", studentName
OUTPUT "Score: ", testScore
```

Example 2 – Selection:

```
INPUT age
IF age >= 18 THEN
   OUTPUT "You can vote"
ELSE
   OUTPUT "Too young to vote"
ENDIF
```

Example 3 – Iteration:

```
total ← 0
FOR count ← 1 TO 5
   INPUT number
   total ← total + number
NEXT count
OUTPUT "Sum is ", total
```

Example 4 – Nested selection:

```
INPUT mark
IF mark >= 70 THEN
   grade ← "A"
ELSE
   IF mark >= 60 THEN
      grade ← "B"
   ELSE
```

```
    IF mark >= 50 THEN
      grade ← "C"
    ELSE
      grade ← "Fail"
    ENDIF
  ENDIF
ENDIF
OUTPUT "Grade: ", grade
```

Important notes about pseudocode:

- Indentation matters (shows which statements are inside blocks)
- ← means "becomes" or "is assigned the value of"
- Use meaningful variable names (score, not s)
- Be consistent with your style

7.2.7 Validation and Verification Design

What is validation?
Validation checks if data is **reasonable** before processing it. It happens automatically by the program.

What is verification?
Verification checks if the data entered **matches the original source**. It often involves a human.

Difference at a glance:

	Validation	**Verification**
Question	Is this data sensible?	Is this data correct?
Example	Age 150 is impossible	Age 25 vs birth certificate says 24
Who does it?	Computer automatically	Human or double-entry
When?	Before processing	Before or after entry

VALIDATION

Six types of validation checks:

Type 1: Range Check

Ensures a number is between a minimum and maximum.

Example – Age input:

```
INPUT age
IF age < 0 OR age > 120 THEN
   OUTPUT "Error: Age must be between 0 and 120"
ELSE
   OUTPUT "Age accepted"
ENDIF
```

Common ranges:

- Month: 1-12
- Day: 1-31 (but also depends on month)
- Percentage: 0-100
- Year: 1900-2025

Type 2: Type Check

Ensures the data type is correct.

Example – Numeric check:

```
INPUT userInput
IF NOT IS_NUMERIC(userInput) THEN
   OUTPUT "Error: Please enter a number"
ENDIF
```

Common type checks:

- Numeric (no letters)
- Alphabetic (no numbers)
- Alphanumeric (letters and numbers allowed)
- Date (valid date format)

Type 3: Length Check

Ensures a string is the correct length.

Example – Password:

```
INPUT password
IF LENGTH(password) < 8 THEN
   OUTPUT "Error: Password must be at least 8 characters"
ENDIF
```

Example – Fixed length (postal code):

```
INPUT postcode
IF LENGTH(postcode) != 5 THEN
   OUTPUT "Error: Postal code must be exactly 5 digits"
ENDIF
```

Type 4: Presence Check

Ensures a field is not left empty.

Example – Required field:

```
INPUT name
IF name = "" THEN
   OUTPUT "Error: Name cannot be blank"
ENDIF
```

Type 5: Format Check (Pattern Check)

Ensures data matches a specific pattern.

Example – Email validation:

```
INPUT email
IF email DOES NOT CONTAIN "@" OR email DOES NOT CONTAIN "." THEN
   OUTPUT "Error: Invalid email format"
ENDIF
```

Example – UK National Insurance number:
Pattern: Two letters, six digits, one letter (AB123456C)

```
IF NOT (letter, letter, digit, digit, digit, digit, digit, digit, letter) THEN
```

```
    OUTPUT "Error: Invalid NI number format"
ENDIF
```

Type 6: Check Digit

A special digit calculated from the other digits to catch typos.

Example – ISBN-13 (book identifier):
The last digit is calculated from the first 12 digits.

How it works:

1. Multiply each of the first 12 digits alternately by 1 and 3
2. Sum the results
3. Find the remainder when divided by 10
4. Subtract from 10 (if result is 10, check digit is 0)

Example ISBN: 978-0-306-40615-?

- Calculate: 9×1 + 7×3 + 8×1 + 0×3 + 3×1 + 0×3 + 6×1 + 4×3 + 0×1 + 6×3 + 1×1 + 5×3
- = 9 + 21 + 8 + 0 + 3 + 0 + 6 + 12 + 0 + 18 + 1 + 15 = 93
- 93 ÷ 10 = remainder 3
- Check digit = 10 - 3 = 7
- Complete ISBN: 978-0-306-40615-7

If someone types 978-0-306-40615-8, the computer recalculates and sees 7 ≠ 8 → error.

VERIFICATION

Two types of verification:

Type 1: Double Entry

User enters data twice; computer compares.

Example – Email signup:

```
INPUT email
INPUT confirmEmail
IF email != confirmEmail THEN
    OUTPUT "Error: Emails do not match"
ENDIF
```

Common uses:

- Password creation
- Email address entry
- Critical financial transactions

Type 2: Proofreading (Visual Check)

User sees what they entered and confirms it's correct.

Example – Order summary:

OUTPUT "Please confirm your order:"

OUTPUT "Name: ", name

OUTPUT "Address: ", address

OUTPUT "Total: $", total

INPUT "Is this correct? (Y/N): ", confirmation

When to use:

- When double-entry is too time-consuming
- When data is non-critical
- After bulk data entry

PHASE 3: CODING (Overview)

Goal: Translate the design (pseudocode/flowchart) into a real programming language.

Key principle: Good design makes coding easy. Bad design makes coding a nightmare.

Example – Translating pseudocode to Python:

Pseudocode:

```
INPUT name
INPUT age
IF age >= 18 THEN
    OUTPUT "Welcome, ", name
ELSE
    OUTPUT "Sorry, too young"
```

ENDIF

Python code:

```
name = input("Enter your name: ")
age = int(input("Enter your age: "))
if age >= 18:
    print("Welcome,", name)
else:
    print("Sorry, too young")
```

We will cover coding in extreme detail in Chapter 8.

PHASE 4: TESTING

Goal: Find and fix errors before the user sees them.

Why testing is critical:

- Buggy software costs money (billions per year globally)
- Bugs can cause injury or death (medical devices, airplanes)
- Fixing bugs after release costs 100× more than fixing during development

7.6.1 Types of Errors (Three Kinds)

ERROR TYPE 1: Syntax Errors

What is a syntax error?
Breaking the grammatical rules of the programming language.

Analogy: "Cat dog the ran" – the words are real, but the order makes no sense.

Examples:

Incorrect code	Syntax error	Correct code
x = 5 (missing colon in Python)	Missing colon	if x == 5:
PRINT "Hello" (uppercase)	Python is case-sensitive	print("Hello")

Incorrect code	Syntax error	Correct code
if x = 5 (single equals)	Assignment instead of comparison	if x == 5
for i in range(10) (missing colon)	Missing colon	for i in range(10):
x = (5 + 3 (unclosed parenthesis)	Missing closing)	x = (5 + 3)

How to find syntax errors:

- The compiler/interpreter stops and shows an error message
- Modern IDEs (Integrated Development Environments) underline errors in red

How to fix syntax errors:

1. Read the error message carefully (line number, description)
2. Check the line before the reported line (error is often earlier)
3. Look for missing quotes, parentheses, colons, indentation

ERROR TYPE 2: Logic Errors

What is a logic error?
The program runs (no crash) but produces the wrong result.

Analogy: Following a recipe but using salt instead of sugar – the cake looks fine but tastes terrible.

Examples:

Incorrect code	Intended	Actual result
average = sum / count-1	sum / count	Off by one
if score = 100 (single =)	if score == 100	Always true (assignment)
while x < 10 (x never changes)	while x < 10: x = x+1	Infinite loop
celsius = fahrenheit * 9/5 - 32	celsius = (fahrenheit-32)*5/9	Wrong conversion

How to find logic errors:

- Trace tables (we will learn these soon)
- Print statements to show variable values at different points
- Step through with a debugger
- Test with known inputs (if input 5 and 5 gives 10, sum works; if gives 9, problem)

How to fix logic errors:

1. Isolate the section that is wrong
2. Check your formulas and conditions
3. Test boundary cases
4. Explain your code to someone else (rubber duck debugging)

ERROR TYPE 3: Runtime Errors

What is a runtime error?
The program crashes during execution.

Analogy: A car that starts fine but the engine explodes after 5 minutes of driving.

Examples:

Operation	Runtime error	Why
x = 10 / 0	Division by zero	Mathematically undefined
arr[5] but array size is 3	Index out of bounds	Memory access violation
int("hello")	Value error	Cannot convert letters to number
open("missing.txt")	File not found	File doesn't exist
Recursion without base case	Stack overflow	Runs out of memory

How to fix runtime errors:

- Use **defensive programming** (check conditions before dangerous operations)

- Add error handling (try/catch blocks)
- Validate user input before using it

Example of defensive programming:

```
INPUT denominator
IF denominator != 0 THEN
   result ← 10 / denominator
   OUTPUT result
ELSE
   OUTPUT "Error: Cannot divide by zero"
ENDIF
```

7.6.2 Test Data (Three Categories)

CATEGORY 1: Normal Data

What is normal data?
Typical, valid inputs that the program is expected to handle.

Purpose: Ensure the program works for everyday use.

Example – Age input (expected range 1-120):
Normal data: 25, 42, 67, 18, 35

CATEGORY 2: Boundary Data

What is boundary data?
Values at the extreme edges of valid range.

Purpose: Off-by-one errors are extremely common. Boundaries catch them.

Example – Age input (valid 1-120):
Boundary data: 1 (minimum), 120 (maximum), also test 0 and 121 (invalid)

General rule: Test at, just below, and just above every boundary.

Example – Percentage (0-100):
Boundary values: -1, 0, 1, 99, 100, 101

CATEGORY 3: Invalid Data

What is invalid data?
Values that should be rejected by validation.

Purpose: Ensure the program handles bad input gracefully (no crashes).

Example – Age input:
Invalid data: -5 (negative), 200 (too high), "abc" (text), "25" (string instead of number), blank

Test data table for age input:

Test case	Input	Expected outcome
Normal 1	25	Accepted, process
Normal 2	42	Accepted, process
Boundary 1	1	Accepted (minimum)
Boundary 2	120	Accepted (maximum)
Boundary 3	0	Rejected (below min)
Boundary 4	121	Rejected (above max)
Invalid 1	-10	Rejected
Invalid 2	"twenty"	Rejected (type error)
Invalid 3	(blank)	Rejected (presence check)

7.6.3 Testing Strategies

BLACK BOX TESTING

What is black box testing?
Testing based solely on inputs and expected outputs. The tester cannot see the code.

Analogy: Testing a vending machine by pressing buttons and seeing what comes out – you don't know how the machine works inside.

When to use:

- Testing finished software

- User acceptance testing
- When code is not available (third-party software)

How to do it:

1. Identify all inputs and expected outputs
2. Create test cases covering normal, boundary, invalid
3. Run the program with each test case
4. Compare actual output to expected output

WHITE BOX TESTING

What is white box testing?
Testing that examines the internal structure of the code.

Analogy: A mechanic opens the engine, checks each part, and knows exactly how it should work.

When to use:

- During development
- Testing critical paths
- Ensuring all code is executed

How to do it:

1. Look at the code's logic (IF statements, loops)
2. Create test cases that execute every possible path
3. Ensure each line of code runs at least once

Example – White box test for:

```
IF x > 0 THEN
   OUTPUT "Positive"
ELSE
   OUTPUT "Non-positive"
ENDIF
```

Need two test cases: x=5 (positive path), x=0 (negative path)

DRY RUNNING

What is a dry run?
Manually executing an algorithm with paper and pencil before running it on a computer.

Why dry run?

- Catches logic errors early (cheaper than debugging on computer)
- Builds understanding of how algorithms work
- Required in many exams (trace tables)

How to dry run:

1. Write down all variables
2. Step through each instruction one by one
3. Update variables as they change
4. Record outputs when they happen

Example dry run – Sum of two numbers:

Algorithm:

01 INPUT a

02 INPUT b

03 sum ← a + b

04 OUTPUT sum

Dry run with a=5, b=7:

Step	a	b	sum	Output
01	5	?	?	-
02	5	7	?	-
03	5	7	12	-
04	5	7	12	12

ALPHA TESTING

What is alpha testing?
Internal testing done by the development team before release.

Who does it: Programmers, testers, QA team
Where: In the development environment
When: Before any external users see the software

BETA TESTING

What is beta testing?
Testing by real users in real conditions before final release.

Who does it: Selected customers or volunteers
Where: Their own environments (different hardware, OS)
When: After alpha testing, before final release

Benefits:

- Finds bugs developers missed
- Tests in real-world conditions
- Gets user feedback on features

PHASE 5: MAINTENANCE

Goal: Keep the software working after release.

Three types of maintenance:

Type	Definition	Example
Corrective	Fixing bugs found after release	User reports crash; you fix it
Adaptive	Adapting to new environments	Update for new Windows version
Perfective	Adding new features	User asks for dark mode; you add it

Maintenance costs: Typically 60-80% of total software cost over lifetime. Good design reduces maintenance.

PHASE 6: EVALUATION

Goal: Assess if the software met requirements.

Evaluation questions:

1. Does the software do what was specified in analysis?
2. Is it user-friendly?
3. Is it efficient (fast enough, uses reasonable memory)?
4. Is it robust (handles errors without crashing)?
5. Is it maintainable (code is readable and modular)?

How to evaluate:

- Compare to requirements specification
- User feedback surveys
- Performance benchmarks
- Code reviews

PART 7.3: PSEUDOCDEE (Complete Reference)

7.3.1 Variables and Constants

Variable:
A named storage location whose value can change during program execution.

Rules for variable names:

- Must start with a letter (a-z, A-Z)
- Can contain letters, numbers, underscore
- Cannot contain spaces
- Cannot be a keyword (IF, WHILE, etc.)
- Case-sensitive (Score and score are different)

Valid names: score, player1, high_score, playerName, x

Invalid names: 1stPlayer (starts with number), my score (space), IF (keyword) suM = a * b

Declaring variables:

DECLARE age : INTEGER

DECLARE name : STRING

DECLARE isPass : BOOLEAN

Assigning values:

age ← 25

name ← "Zaki"

isPass ← TRUE

Constants:
A named value that never changes during execution.

CONANT PI ← 3.14159

CONSTANT MAX_SCORE ← 100

7.3.2 Data Types (Detailed)

INTEGER

- Whole numbers (no decimal point)
- Range depends on memory (typically -2,147,483,648 to 2,147,483,647)
- Examples: -42, 0, 7, 1000

REAL (or FLOAT)

- Numbers with decimal points
- Examples: 3.14, -0.001, 2.0, 99.99

CHAR

- A single character (letter, digit, symbol)
- Written in single quotes
- Examples**: 'A', '9', '$', ' ' (space)**

STRING

- A sequence of characters (zero or more)
- Written in double quotes
- Examples: **"Hello", "123", "a", "" (empty string)**

BOOLEAN

- Logical values: TRUE or FALSE
- Used for conditions and flags

DATE

- A calendar date

- Examples: **2025-03-30, 30/03/2025**

7.3.3 Input and Output

Input:

INPUT variableName

INPUT "Prompt text: ", variableName

Output:

OUTPUT expression

OUTPUT "Text ", variable, " more text"

Examples:

OUTPUT "Enter your age: "

INPUT age

OUTPUT "You are ", age, " years old"

7.3.4 Arithmetic Operators

Operator	Operation	Example	Result
+	Addition	5 + 3	8
-	Subtraction	5 - 3	2
*	Multiplication	5 * 3	15
/	Division (real)	5 / 2	2.5
DIV	Integer division	5 DIV 2	2
MOD	Modulo (remainder)	5 MOD 2	1

Order of operations (PEMDAS/BODMAS):

1. Parentheses ()

2. Exponents (not common in basic pseudocode)
3. Multiplication and Division (*, /, DIV, MOD) – left to right
4. Addition and Subtraction (+, -) – left to right

Examples:

3 + 4 * 2 → 3 + 8 = 11

(3 + 4) * 2 → 7 * 2 = 14

10 / 2 + 3 → 5 + 3 = 8

10 / (2 + 3) → 10 / 5 = 2

17 MOD 5 → 2 (because 5*3=15, remainder 2)

Common uses of MOD:

- Check if a number is even: num MOD 2 = 0
- Check if a number is odd: num MOD 2 = 1
- Extract last digit: num MOD 10
- Wrap around (clock arithmetic): hours MOD 12

7.3.5 Relational Operators (for comparisons)

Operator	Meaning	Example	Result (if x=5)
=	Equal to	x = 5	TRUE
<> or !=	Not equal	x <> 5	FALSE
<	Less than	x < 10	TRUE
>	Greater than	x > 10	FALSE
<=	Less than or equal	x <= 5	TRUE
>=	Greater than or equal	x >= 10	FALSE

7.3.6 Logical Operators

Operator	Meaning	Truth table
AND	Both must be TRUE	T AND T = T; T AND F = F; F AND T = F; F AND F = F
OR	At least one must be TRUE	T OR T = T; T OR F = T; F OR T = T; F OR F = F
NOT	Reverses truth value	NOT T = F; NOT F = T

Examples:

```
IF age >= 18 AND hasLicense = TRUE THEN
   OUTPUT "Can drive"
ENDIF

IF day = "Saturday" OR day = "Sunday" THEN
   OUTPUT "Weekend!"
ENDIF

IF NOT isRaining THEN
   OUTPUT "Go outside"
ENDIF
```

7.3.7 String Operators and Functions

Concatenation (joining strings):
Using + or &:

```
fullName ← firstName + " " + lastName
```

Common string functions:

Function	Description	Example	Result
LENGTH(str)	Number of characters	LENGTH("Hello")	5
SUBSTRING(str, start, length)	Extract part	SUBSTRING("Hello",2,3)	"ell"
UPPER(str)	Convert to uppercase	UPPER("Hello")	"HELLO"
LOWER(str)	Convert to lowercase	LOWER("Hello")	"hello"
POSITION(sub, str)	Find position (1-based)	POSITION("ll","Hello")	3

Note: Some pseudocode standards use 0-based indexing. Confirm with your exam board.

7.3.8 SELECTION (IF and CASE)

IF-THEN:

```
IF condition THEN
    statements
ENDIF
```

IF-THEN-ELSE:

```
IF condition THEN
    statements
ELSE
    statements
ENDIF
```

Nested IF (ELSE IF):

```
IF condition1 THEN
```

statements

ELSE IF condition2 THEN

statements

ELSE IF condition3 THEN

statements

ELSE

statements

ENDIF

CASE (Switch) statement:
Use when comparing one variable to many possible values.

CASE variable OF

value1: statements

value2: statements

value3: statements

OTHERWISE: statements

ENDCASE

Example:

CASE grade OF

"A": OUTPUT "Excellent"

"B": OUTPUT "Good"

"C": OUTPUT "Fair"

"D": OUTPUT "Poor"

OTHERWISE: OUTPUT "Fail"

ENDCASE

7.3.9 ITERATION (Loops)

FOR loop (count-controlled, definite):
Used when you know exactly how many times to repeat.

FOR counter ← start TO end [STEP increment]

statements

```
NEXT counter
```

Examples:

```
FOR i ← 1 TO 10
   OUTPUT i
NEXT i
```

Outputs: 1,2,3,4,5,6,7,8,9

text

```
FOR i ← 10 DOWNTO 1
   OUTPUT i
NEXT i
```

Outputs: 10,9,8,7,6,5,4,3,2,1

text

```
FOR i ← 1 TO 10 STEP 2
   OUTPUT i
NEXT i
```

Outputs: 1,3,5,7,9

WHILE loop (pre-test condition, indefinite):
Checks condition BEFORE each iteration. May run zero times.

```
WHILE condition DO
   statements
ENDWHILE
```

Example – Keep asking until valid input:

```
INPUT age
WHILE age < 0 OR age > 120 DO
   OUTPUT "Invalid. Enter age 0-120: "
   INPUT age
ENDWHILE
```

REPEAT UNTIL loop (post-test condition, indefinite):
Checks condition AFTER each iteration. Always runs at least once.

```
REPEAT
    statements
UNTIL condition
```

Example – Menu system:

```
REPEAT
    OUTPUT "1. Add"
    OUTPUT "2. Subtract"
    OUTPUT "3. Exit"
    INPUT choice
    IF choice = 1 THEN CALL add()
    IF choice = 2 THEN CALL subtract()
UNTIL choice = 3
```

WHILE vs REPEAT UNTIL:

WHILE	REPEAT UNTIL
Checks before	Checks after
May run 0 times	Runs at least once
Condition is "continue while true"	Condition is "stop when true"

7.3.10 SUBROUTINES (Procedures and Functions)

A subroutine is a named, reusable block of code designed to perform a specific task within a larger program, often called a function, procedure, method, or subprogram.

It improves code efficiency, readability, and organization by reducing duplication and enabling complex tasks to be broken down into smaller, manageable

Procedure:
Performs a task, may have parameters, does NOT return a value.

```
PROCEDURE procedureName(parameters)
    statements
```

```
ENDPROCEDURE
```

Calling a procedure:

```
CALL procedureName(arguments)
```

Example:

```
PROCEDURE greet(name)
    OUTPUT "Hello, ", name
ENDPROCEDURE


CALL greet("Zaki")    // Outputs: Hello, Zaki
```

Function:
Performs a task, returns a single value.

```
FUNCTION functionName(parameters) RETURNS dataType
    statements
    RETURN value
ENDFUNCTION
```

Calling a function:

```
variable ← functionName(arguments)
```

Example:

```
FUNCTION double(x) RETURNS INTEGER
    RETURN x * 2
ENDFUNCTION


result ← double(5)   // result = 10
```

Parameters and arguments:

- **Parameter:** Variable in the subroutine definition (placeholder)
- **Argument:** Actual value passed when calling

Pass by value (default):

- A copy of the argument is made
- Changes inside subroutine do NOT affect original

Pass by reference (if specified):

- The original variable is used
- Changes inside subroutine DO affect original
- Use BYREF keyword

text

```
PROCEDURE changeValue(BYREF x)
   x ← 100
ENDPROCEDURE


num ← 5
CALL changeValue(num)   // num becomes 100
```

PART 7.4: COMMON ALGORITHMS

7.4.1 LINEAR SEARCH

Linear Search Algorithm (also called *Sequential Search*) is one of the simplest searching techniques.

What it does

It checks each element in a list one by one until:

- the target element is found, or
- the list ends.

Problem: Find the position of a target value in an **unsorted list**.

How it works:

1. Start at the first element
2. Compare each element to the target,
3. If it matches → return the position.

   If found, return the position,
4. If not → move to the next element.
5. Repeat until found or list ends.

6. If reach the end without finding, return -1 (not found)

Time complexity: O(n) – in worst case, checks all n elements.

Pseudocode:

```
FUNCTION linearSearch(arr, target) RETURNS INTEGER
  FOR i ← 0 TO LENGTH(arr) - 1
    IF arr[i] = target THEN
      RETURN i
    ENDIF
  NEXT i
  RETURN -1
ENDFUNCTION
```

Example dry run:
Array = [45, 23, 67, 12, 89], target = 67

i	arr[i]	arr[i]=target?	Action
0	45	No	Continue
1	23	No	Continue
2	67	Yes	Return 2

When to use:

- Small lists (n < 100)
- Unsorted lists
- List changes frequently (sorting overhead not worth it)

When NOT to use:

- Large lists (binary search is faster)
- When you need many searches (sort once, then binary search)

7.4.2 BINARY SEARCH

Problem: Find the position of a target value in a **SORTED** list.

How it works:

1. **Find the middle element**
2. **If target equals middle, return position**
3. **If target < middle, search left half**
4. **If target > middle, search right half**
5. **Repeat until found or no elements remain**

Requirement: List MUST be sorted.

Time complexity: O(log n) – each step eliminates half the list.

Pseudocode:

```
FUNCTION binarySearch(arr, target) RETURNS INTEGER
    low ← 0
    high ← LENGTH(arr) - 1

    WHILE low <= high DO
        mid ← (low + high) DIV 2

        IF arr[mid] = target THEN
            RETURN mid
        ELSE IF arr[mid] < target THEN
            low ← mid + 1
        ELSE
            high ← mid - 1
        ENDIF
    ENDWHILE

    RETURN -1
ENDFUNCTION
```

Example dry run:
Sorted array = [12, 23, 45, 67, 89], target = 67

low	high	mid	arr[mid]	Comparison	New low/high
0	4	2	45	67 > 45	low = 3
3	4	3	67	67 = 67	Return 3

Binary search visualized:

[12, 23, 45, 67, 89]

↑

middle (45) → target 67 is greater, search right half

[67, 89]

↑

middle (67) → found!

Comparison: Linear vs Binary on 1 million items:

- Linear: up to 100 comparisons
- Binary: up to 20 comparisons (because $\log_2 1{,}000{,}000 \approx 20$)

7.4.3 BUBBLE SORT

Problem: Sort a list into ascending order.

How it works:

1. Compare adjacent elements
2. If they are in wrong order, swap them
3. Repeat passes until no swaps needed
4. After each pass, the largest element "bubbles" to the end

Time complexity: $O(n^2)$ – very slow for large lists.

Pseudocode:

```
PROCEDURE bubbleSort(arr)
    n ← LENGTH(arr)
```

```
  FOR i ← 0 TO n-2
    swapped ← FALSE

    FOR j ← 0 TO n-i-2
      IF arr[j] > arr[j+1] THEN
        // Swap
        temp ← arr[j]
        arr[j] ← arr[j+1]
        arr[j+1] ← temp
        swapped ← TRUE
      ENDIF
    NEXT j

    // If no swaps, list is sorted
    IF swapped = FALSE THEN
      EXIT
    ENDIF
  NEXT i
ENDPROCEDURE
```

Example dry run:
Array = [64, 34, 25, 12, 22]

Pass 1:

Compare	Swap?	Array after
64,34	Yes	[34,64,25,12,22]
64,25	Yes	[34,25,64,12,22]

Compare	Swap?	Array after
64,12	Yes	[34,25,12,64,22]
64,22	Yes	[34,25,12,22,64]

Pass 2:

Compare	Swap?	Array after
34,25	Yes	[25,34,12,22,64]
34,12	Yes	[25,12,34,22,64]
34,22	Yes	[25,12,22,34,64]

Pass 3:

Compare	Swap?	Array after
25,12	Yes	[12,25,22,34,64]
25,22	Yes	[12,22,25,34,64]

Pass 4:

Compare	Swap?	Array after
12,22	No	[12,22,25,34,64]

Final sorted array: [12, 22, 25, 34, 64]

Why bubble sort is inefficient:
For 10,000 items: ~50 million comparisons
For 1,000,000 items: ~500 billion comparisons

When to use bubble sort:

- Very small lists (n < 100)
- Teaching sorting concepts
- When list is almost sorted (can finish early)

7.4.4 FINDING MAXIMUM AND MINIMUM

Find maximum:

```
FUNCTION findMax(arr) RETURNS INTEGER
   max ← arr[0]
   FOR i ← 1 TO LENGTH(arr)-1
      IF arr[i] > max THEN
         max ← arr[i]
      ENDIF
   NEXT i

   RETURN max
ENDFUNCTION
```

Find minimum:

text

```
FUNCTION findMin(arr) RETURNS INTEGER
   min ← arr[0]

   FOR i ← 1 TO LENGTH(arr)-1
      IF arr[i] < min THEN
         min ← arr[i]
      ENDIF
   NEXT i

   RETURN min
ENDFUNCTION
```

Time complexity: O(n)

7.4.5 COUNTING OCCURRENCES

Count how many times a target appears:

```
FUNCTION countOccurrences(arr, target) RETURNS INTEGER
    count ← 0

    FOR i ← 0 TO LENGTH(arr)-1
        IF arr[i] = target THEN
            count ← count + 1
        ENDIF
    NEXT i

    RETURN count
ENDFUNCTION
```

7.4.6 CALCULATING SUM AND AVERAGE

Sum:

```
FUNCTION sum(arr) RETURNS INTEGER
    total ← 0

    FOR i ← 0 TO LENGTH(arr)-1
        total ← total + arr[i]
    NEXT i

    RETURN total
ENDFUNCTION
```

Average:

```
FUNCTION average(arr) RETURNS REAL
    IF LENGTH(arr) = 0 THEN
```

```
    RETURN 0
  ENDIF

  total ← 0
  FOR i ← 0 TO LENGTH(arr)-1
    total ← total + arr[i]
  NEXT i

  RETURN total / LENGTH(arr)
ENDFUNCTION
```

7.4.7 REVERSING AN ARRAY

text

```
PROCEDURE reverseArray(arr)
  n ← LENGTH(arr)
  FOR i ← 0 TO (n DIV 2) - 1
    // Swap element i with element n-1-i
    temp ← arr[i]
    arr[i] ← arr[n-1-i]
    arr[n-1-i] ← temp
  NEXT i
ENDPROCEDURE
```

Example:
Array = [1,2,3,4,5]
After reverse = [5,4,3,2,1]

7.4.8 CHECKING FOR PALINDROME

What is a palindrome?
A string that reads the same forwards and backwards.
Examples: "radar", "level", "racecar", "A man a plan a canal panama" (ignoring spaces)

Pseudocode (case-sensitive, spaces matter):

```
FUNCTION isPalindrome(str) RETURNS BOOLEAN
    n ← LENGTH(str)

    FOR i ← 0 TO (n DIV 2) - 1
        IF str[i] <> str[n-1-i] THEN
            RETURN FALSE
        ENDIF
    NEXT i

    RETURN TRUE
ENDFUNCTION
```

Example dry run: str = "radar", n=5

I	str[i]	str[4-i]	Match?
0	'r'	'r'	Yes
1	'a'	'a'	Yes
Loop ends (i=2 stops because 5/2=2) → Return TRUE			

PART 7.5: TRACE TABLES

7.5.1 What is a Trace Table?

A **trace table** is a technique to manually simulate an algorithm's execution by tracking how variable values change step by step.

Why use trace tables:

- Find logic errors without running code
- Understand how complex algorithms work
- Required skill in many computer science exams
- Debug when you cannot use a computer

7.5.2 How to Create a Trace Table

Step 1: List all variables that change (including loop counters)
Step 2: Add a column for outputs (if any)
Step 3: Add a column for conditions (if needed for clarity)
Step 4: Execute each line of the algorithm in order
Step 5: Record changes after each instruction

7.5.3 Trace Table Examples

Example 1 – Simple sequence; Add 5, 10, 15, 20

01 a ← 5

02 b ← 10

03 c ← a + b

04 OUTPUT c

Trace table:

Step	A	B	C	D	C = A + B + C + D	Output
1	5					
2	5	10				
3	5	10	15			
4	5	10	15	20		
5	5	10	15	20	40	
6						40

Explanation

- **Step 1: Assign a = 5**
- **Step 2: Assign b = 10**

- **Step 3: Compute c = a + b = 15**
- **Step 4: Output c**

Example 2 – IF statement:

```
01  INPUT score
02  IF score >= 70 THEN
03      grade ← "A"
04  ELSE
05      grade ← "Fail"
06  ENDIF
07  OUTPUT grade
```

Test case 1: score = 85

Step	score	grade	Output	Condition (score>=70)
1	85			
2	85			TRUE
3	85	"A"		
4	85	"A"	"A"	

Test case 2: score = 45

Step	score	grade	Output	Condition (score>=70)
01	45			
02	45			FALSE
05	45	"Fail"		

Step	score	grade	Output	Condition (score>=70)
07	45	"Fail"	"Fail"	

Example 3 – FOR loop:

1 total ← 0

2 FOR i ← 1 TO 3

3 INPUT num

4 total ← total + num

5 NEXT i

6 OUTPUT total

Inputs: 10, 20, 30

Step	total	i	num	Output	Notes
01	0				
02	0	1			Enter loop
03	0	1	10		
04	10	1	10		
05	10	2	10		NEXT i
03	10	2	20		
04	30	2	20		
05	30	3	20		NEXT i

Step	total	i	num	Output	Notes
03	30	3	30		
04	60	3	30		
05	60	4	30		Loop ends (i > 3)
06	60	4	30	60	

Example 4 – WHILE loop:

```
1  count ← 0
02  number ← 1
03  WHILE count < 3 DO
04      OUTPUT number
05      number ← number + 2
06      count ← count + 1
07  ENDWHILE
```

Trace table:

Step	count	number	Output	Condition (count<3)
01	0			
02	0	1		
03	0	1		TRUE (0<3)
04	0	1	1	

Step	count	number	Output	Condition (count<3)
05	0	3		
06	1	3		
07	1	3		Loop back
03	1	3		TRUE (1<3)
04	1	3	3	
05	1	5		
06	2	5		
07	2	5		Loop back
03	2	5		TRUE (2<3)
04	2	5	5	
05	2	7		
06	3	7		
07	3	7		Loop back
03	3	7		FALSE (3<3) Exit

Output: 1, 3, 5

Example 5 – Nested loops:

text

```
01  FOR i ← 1 TO 2
02      FOR j ← 1 TO 3
03          OUTPUT i * j
04      NEXT j
05  NEXT i
```

Trace table:

Step	i	j	Output
01	1		
02	1	1	
03	1	1	1
04	1	2	
03	1	2	2
04	1	3	
03	1	3	3
04	1	4	(j loop ends)
05	2	4	
02	2	1	
03	2	1	2
04	2	2	

Step	i	j	Output
03	2	2	4
04	2	3	
03	2	3	6
04	2	4	(j loop ends)
05	3	4	(i loop ends)

Output: 1,2,3,2,4,6

7.5.4 Common Trace Table Mistakes

Mistake 1: Forgetting to update variables after each step
Solution: Update immediately after the line executes

Mistake 2: Losing track of loop counters
Solution: Write the counter value after NEXT/ENDWHILE

Mistake 3: Not recording outputs in order
Solution: Add an Output column and record in sequence

Mistake 4: Confusing assignment (=) with equality test
Solution: Assignment changes value; equality test does not

PART 7.6: FINDING THE PURPOSE OF AN ALGORITHM

7.6.1 Strategy

When given an algorithm and asked "What does it do?":

1. **Identify inputs** – What data goes in?
2. **Identify outputs** – What comes out?
3. **Look for key patterns:**
    - Summing → total = total + x
    - Counting → count = count + 1
    - Finding max/min → comparisons

- Searching → looking for a match
- Sorting → swapping adjacent elements

4. **Trace with sample data** – Run a small example
5. **Generalize** – Describe the purpose in one sentence

7.6.2 Examples

Example 1:

text

```
INPUT n
result ← 1
FOR i ← 1 TO n
   result ← result * i
NEXT i
OUTPUT result
```

Trace with n=4:
i=1: result=1×1=1
i=2: result=1×2=2
i=3: result=2×3=6
i=4: result=6×4=24
Output: 24

Purpose: Calculates factorial of n (n! = 1×2×3×...×n)

Example 2:

text

```
INPUT text
count ← 0
FOR i ← 1 TO LENGTH(text)
   char ← SUBSTRING(text, i, 1)
   IF char = "a" OR char = "e" OR char = "i" OR char = "o" OR char = "u" THEN
      count ← count + 1
   ENDIF
NEXT i
```

```
OUTPUT count
```

Trace with text="hello":
h→no, e→yes(count=1), l→no, l→no, o→yes(count=2)
Output: 2

Purpose: Counts the number of vowels (a,e,i,o,u) in a string.

Example 3:

```
text

INPUT a, b

WHILE a <> b DO

   IF a > b THEN

      a ← a - b

   ELSE

      b ← b - a

   ENDIF

ENDWHILE

OUTPUT a
```

Trace with a=12, b=18:
12≠18: a=12, b=18 → a<b → b=18-12=6
12≠6: a=12, b=6 → a>b → a=12-6=6
6=6: loop ends, output 6

Purpose: Finds the greatest common divisor (GCD) of a and b using Euclidean algorithm.

PART 7.7: FINDING AND CORRECTING ERRORS

7.7.1 Systematic Debugging Process

Step 1: Reproduce the error consistently
Step 2: Isolate where it happens (which line/function)
Step 3: Check assumptions (variables have expected values?)
Step 4: Fix one error at a time
Step 5: Retest after each fix

7.7.2 Common Errors and Fixes

Error Type	Example	Fix
Off-by-one	FOR i=1 TO n-1 should be TO n	Adjust loop bounds
Uninitialized variable	total = total + x (total starts unknown)	Initialize: total=0
Infinite loop	WHILE x<10 (x never changes)	Add x=x+1 inside
Wrong comparison	IF x=5 (assignment)	Use IF x==5
Wrong operator	average = sum/2 (should divide by count)	average = sum/coun
Missing ENDIF	IF x>5 without closing	Add ENDIF
Array index out of bounds	arr[10] when size is 10 (valid 0-9)	Use arr[9]

7.7.3 Debugging Techniques

Print statement debugging:
Insert temporary outputs to see variable values.

```
total ← 0
FOR i ← 1 TO 5
    total ← total + i
    OUTPUT "i=", i, " total=", total   // Debug line
NEXT i
```

Rubber duck debugging:
Explain your code line by line to an inanimate object (rubber duck). The act of explaining often reveals the error.

Divide and conquer:
Comment out half the code. If error disappears, it's in that half. Repeat.

END OF CHAPTER 7

VOLUME 2

CHAPTER 8: PROGRAMMING

Areas of Study:-

- Programming concepts (8.1)
- Data types (8.2) – including type conversion, overflow, underflow
- Input and output (8.3) – formatting, validation loops
- Arithmetic operators (8.4) – precedence, integer division tricks
- Sequence (8.5)
- Selection (8.6) – nested IF, CASE, conditional expressions
- Iteration (8.7) – FOR, WHILE, REPEAT, nested loops, loop invariants
- Totalling (8.8) – running totals, accumulators
- Counting (8.9) – counters, frequency arrays
- String manipulation (8.10) – all functions, pattern matching
- Nested statements (8.11) – complex examples
- Subroutines (8.12) – scope, parameters, recursion
- Library routines (8.13) – common libraries
- Maintainable programs (8.14) – style guides, documentation
- Arrays (8.15) – 1D, 2D, dynamic arrays, array algorithms
- File handling (8.16) – read, write, append, CSV, binary

PART 8.1: PROGRAMMING CONCEPTS (The Absolute Beginning)

8.1.1 What is a Program?

Simple definition: A program is a set of instructions that tells a computer what to do.

Analogy: A recipe is to a chef what a program is to a computer. The recipe lists steps (instructions), ingredients (data), and equipment (resources). The chef follows the recipe exactly to create a dish. The computer follows the program exactly to produce a result.

Key insight: Computers are incredibly fast but incredibly stupid. They do exactly what you tell them, nothing more, nothing less. If you tell them to do the wrong thing, they will do the wrong thing at lightning speed.

8.1.2 The Difference Between Data and Instructions

	Data	Instructions
What it is	Information being processed	Commands telling what to do
Example	42, "Hello", 3.14	x = x + 1, IF age > 18
Can change?	Yes (variables)	No (fixed during execution)
Storage	Variables, arrays, files	The program code itself

8.1.3 Variables

What is a variable?
A named storage location in the computer's memory that holds a value. The value can change during program execution.

Analogy: A variable is like a labeled box. You can put things in the box, take things out, or replace what's inside with something else.

Visual representation:

Memory location: [42]

↑

Variable name: age

Why do we need variables?

1. **Temporary storage** – Keep intermediate results
2. **Flexibility** – Same code works with different inputs
3. **Readability** – Names like total are clearer than memory addresses

Variable naming rules (most languages):

Rule	Valid	Invalid
Start with letter or underscore	score, _temp	1stPlace
Can contain letters, numbers, underscore	player1, high_score	player-name

Rule	Valid	Invalid
No spaces	firstName	first name
Not a keyword	total	if, while, for, print
Case-sensitive	Score ≠ score	N/A

Naming conventions (best practices):

Convention	Example	When to use
camelCase	studentAge, totalScore	Variables (most languages)
snake_case	student_age, total_score	Variables (Python, Ruby)
PascalCase	StudentAge, TotalScore	Classes, types
UPPER_CASE	MAX_SCORE, PI	Constants

8.1.4 Constants

What is a constant?
A named storage location whose value CANNOT change after it is set.

Why use constants?

1. **Safety** – Prevents accidental modification
2. **Clarity** – PI is clearer than 3.14159265359
3. **Maintainability** – Change in one place, not everywhere

Example without constant (BAD):

area = radius * 3.14159 * 3.14159 // What is 3.14159?

circumference = 2 * 3.14159 * radius // Same number repeated

Example with constant (GOOD):

text

CONSTANT PI ← 3.14159

area = radius * PI * PI x = x +1

circumference = 2 * PI * radius

8.1.5 Assignment (Deep Dive)

What is assignment?
Giving a value to a variable. The assignment operator is usually = or ←.

Syntax: variable ← expression

Important: The equals sign in programming is NOT the same as in mathematics.

Mathematics	**Programming**
x = 5 means x is always 5	x = 5 means put 5 into x
x = x + 1 is impossible (5=6?)	x = x + 1 is common (take current x, add 1, store back)

Reading assignment correctly:
total = total + 5
Read as: "Calculate the value of total + 5, then store that result into the variable total"

Step-by-step execution of x = x + 1 **when x is currently 5:**

1. Look at right side: x + 1 → 5 + 1 → 6
2. Store the result (6) into the variable x
3. Now x contains 6

Types of assignment:

Type	Example	Meaning
Direct	x = 5	Store the number 5
Copy	y = x	Copy x's value into y

Type	Example	Meaning
Calculation	total = price + tax	Store the result of calculation
Increment	count = count + 1	Increase by 1
Decrement	remaining = remaining - 1	Decrease by 1

8.1.6 Declaration vs Initialization

Declaration: Telling the computer that a variable exists and what type it is.
Initialization: Giving a variable its first value.

Not initialized (dangerous):

DECLARE score: INTEGER

OUTPUT score // What value? Unknown! Could be anything.

Initialized (safe):

DECLARE score : INTEGER

score ← 0 // Now we know it's 0

Declaration and initialization together:

score ← 0

or in some languages:

text

DECLARE score : INTEGER ← 0

Uninitialized variable errors:

- Value is whatever was left in memory from previous program
- Different every time you run
- Extremely hard to debug
- **Rule:** Always initialize variables before using them

8.1.7 Variable Scope

What is scope?
The region of a program where a variable is accessible.

Two types of scope:

1. Local Scope

- Declared inside a subroutine (procedure or function)
- Only accessible inside that subroutine
- Created when subroutine starts, destroyed when subroutine ends
- Different subroutines can have same variable names (they are separate)

Example:

```
PROCEDURE calculateSum()
   DECLARE x : INTEGER   // Local variable x
   x ← 5
   OUTPUT x              // Works fine
ENDPROCEDURE

PROCEDURE calculateProduct()
   DECLARE x : INTEGER   // Different local variable x
   x ← 10
   OUTPUT x              // Works fine, outputs 10
ENDPROCEDURE

CALL calculateSum()       // Outputs 5
CALL calculateProduct()   // Outputs 10
// The two x variables are completely separate!
```

2. Global Scope

- Declared outside any subroutine
- Accessible everywhere in the program
- Exists for entire program execution

Example:

text

```
DECLARE globalCounter : INTEGER  // Global variable

PROCEDURE increment()
    globalCounter ← globalCounter + 1   // Can access
ENDPROCEDURE

PROCEDURE decrement()
    globalCounter ← globalCounter - 1   // Can access
ENDPROCEDURE

globalCounter ← 0
CALL increment()   // globalCounter becomes 1
CALL decrement()   // globalCounter becomes 0
```

Global vs Local comparison:

Aspect	Global	Local
Access	Everywhere	Only inside its subroutine
Lifetime	Whole program	While subroutine runs
Memory usage	Always占用	Temporary
Risk of bugs	High (unexpected changes)	Low (isolated)
When to use	Rarely	Usually

Best practice: Use local variables whenever possible. Only use global for truly shared data (configuration settings, constants).

PART 8.2: DATA TYPES

8.2.1 What is a Data Type?

Definition: A data type specifies what kind of value a variable can hold and what operations can be performed on it.

Why data types matter:

- **Memory allocation** – Different types need different amounts of memory
- **Operations allowed** – Can't divide text or capitalize numbers
- **Error prevention** – Catches mistakes like adding a name to a number

8.2.2 The Five Primary Data Types

TYPE 1: INTEGER

What it is: Whole numbers (no decimal point), can be positive, negative, or zero.

Examples: -42, -1, 0, 1, 7, 1000, 999999

Memory size: Usually 4 bytes (32 bits) or 8 bytes (64 bits)

Range (4 bytes): -2,147,483,648 to 2,147,483,647

Range (8 bytes): -9,223,372,036,854,775,808 to 9,223,372,036,854,775,807

Operations allowed: +, -, *, /, DIV, MOD, <, >, =, etc.

When to use:

- Counting things (number of students, age in years)
- Index positions in arrays
- Loop counters
- Any whole number

Common integer operations:

Operation	Example	Result
Addition	7 + 3	10
Subtraction	7 - 3	4
Multiplication	7 * 3	21

Operation	Example	Result
Division (real)	7 / 3	2.333... (but careful with integer division)
Integer division	7 DIV 3	2 (quotient)
Modulo	7 MOD 3	1 (remainder)

Integer overflow:
What happens when you exceed the maximum value?

max = 2,147,483,647

max + 1 = -2,147,483,648 (wraps around!)

This is called **overflow**. Different languages handle it differently:

- Some crash (runtime error)
- Some wrap around silently
- Some promote to larger type

Preventing overflow:

- Choose appropriate size (use 64-bit for large numbers)
- Check before operations: IF x > MAX_VALUE - y THEN error
- Use libraries for arbitrary-precision integers

TYPE 2: REAL (also called FLOAT or DOUBLE)

What it is: Numbers with decimal points (floating point numbers).

Examples: 3.14, -0.001, 2.0, 99.99, 1.2345e-10 (scientific notation)

Memory size:

- Single precision (FLOAT): 4 bytes, about 7 decimal digits precision
- Double precision (DOUBLE): 8 bytes, about 15-16 decimal digits precision

Range: Approximately $\pm 1.18 \times 10^{-38}$ to $\pm 3.4 \times 10^{38}$ (float), much larger for double

Operations allowed: +, -, *, /, <, >, =, etc. (but be careful with equality)

When to use:

- Measurements (height, weight, temperature)
- Money (but integers in cents are often better)
- Scientific calculations
- Any number that needs fractional parts

Precision limitations (critical to understand):
Some numbers cannot be represented exactly in binary floating point.

Example:

text

0.1 + 0.2 = 0.30000000000000004 (not exactly 0.3!)

Why? Just as 1/3 cannot be represented exactly in decimal (0.33333...), 0.1 cannot be represented exactly in binary.

Consequences:

- Never compare floating point numbers with =
- Use a tolerance: IF ABS(a - b) < 0.00001 THEN they are equal

Example of correct comparison:

text

```
// BAD:
IF a = 0.1 THEN ...

// GOOD:
IF ABS(a - 0.1) < 0.0000001 THEN ...
```

Converting between integer and real:

text

```
// Integer to real (automatic, safe)
intValue ← 5
realValue ← intValue    // realValue becomes 5.0

// Real to integer (truncation, loses decimal)
realValue ← 7.8
```

intValue ← REAL_TO_INT(realValue) // intValue becomes 7 (not 8!)

Rounding functions:

Function	Input 7.4	Input 7.5	Input 7.6
ROUND	7	8	8
FLOOR (round down)	7	7	7
CEILING (round up)	8	8	8
TRUNCATE (remove decimal)	7	7	7

TYPE 3: CHAR (Character)

What it is: A single character: letter, digit, punctuation, space, or control character.

Examples: 'A', 'a', '9', '$', ' ' (space), '\n' (newline)

Memory size: 1 byte (ASCII) or 2-4 bytes (Unicode)

How characters are stored: Characters are stored as numbers using a character encoding.

ASCII (American Standard Code for Information Interchange):

- 7-bit encoding (0-127)
- Defines English letters, digits, punctuation, control characters

Common ASCII values (MEMORIZE THESE):

Character	ASCII Value (Decimal)
'0'	48
'1'	49
...	...

Character	ASCII Value (Decimal)
'9'	57
'A'	65
'B'	66
...	...
'Z'	90
'a'	97
'b'	98
...	...
'z'	122
Space	32
Newline	10

Why ASCII matters:

- Characters can be compared: 'A' < 'B' is TRUE (65 < 66)
- Uppercase and lowercase are different: 'A' (65) vs 'a' (97)
- Digits are sequential: '0' (48) to '9' (57)

Converting between char and integer:

text

// Char to integer (get ASCII code)

charValue ← 'A'

```
code ← CHAR_TO_INT(charValue)   // code = 65


// Integer to char (convert ASCII code to character)
code ← 65
charValue ← INT_TO_CHAR(code)   // charValue = 'A'


// Convert digit char to its numeric value
digitChar ← '7'
numericValue ← CHAR_TO_INT(digitChar) - 48   // 55 - 48 = 7
```

Unicode: Extended character set for all languages (emoji, Chinese, Arabic, etc.). First 128 characters are the same as ASCII.

TYPE 4: STRING

What it is: A sequence of zero or more characters.

Examples: "" (empty string), "H", "Hello", "Hello World!", "123" (still a string, not a number)

Memory size: Variable (each character typically 1-4 bytes + overhead)

String operations:

Operation	Example	Result
Concatenation (join)	"Hello" + " " + "World"	"Hello World"
Length	LENGTH("Hello")	5
Character access (indexing)	"Hello"[1]	'e' (0-based)
Substring	SUBSTRING("Hello",2,3)	"ell"
Find position	POSITION("ll","Hello")	3
Uppercase	UPPER("Hello")	"HELLO"

Operation	Example	Result
Lowercase	LOWER("Hello")	"hello"

Important: String indexing
Most languages use 0-based indexing (first character is at position 0). Some pseudocode standards use 1-based. Check your exam board.

0-based indexing (Python, Java, C, C++, JavaScript):

text

"Hello"

Index: 0:H, 1:e, 2:l, 3:l, 4:o

1-based indexing (pseudocode standards, VB.NET, Lua):

text

"Hello"

Index: 1:H, 2:e, 3:l, 4:l, 5:o

Empty string:

text

```
empty ← ""
IF empty = "" THEN
   OUTPUT "String is empty"
ENDIF
```

String comparison:

- Strings are compared character by character using ASCII/Unicode values
- "Apple" < "Banana" is TRUE ('A' (65) < 'B' (66))
- "apple" > "Apple" is TRUE ('a' (97) > 'A' (65))
- Comparison is case-sensitive unless you convert to same case

String escape sequences (special characters):

Sequence	Meaning
\n	Newline
\t	Tab
\"	Double quote inside double-quoted string
\'	Single quote inside single-quoted string
\\	Backslash character

Example:

text

OUTPUT "Line1\nLine2\nLine3"

Outputs:

text

Line1

Line2

Line3

TYPE 5: BOOLEAN

What it is: Logical true/false values.

Possible values: TRUE or FALSE (sometimes 1 and 0 in older languages)

Memory size: Usually 1 byte (but only 1 bit is needed)

Operations allowed:

- AND (both must be TRUE)
- OR (at least one must be TRUE)
- NOT (reverse)
- XOR (exclusive OR: exactly one is TRUE)

Truth tables (MEMORIZE THESE):

AND:

A	B	A AND B
F	F	F
F	T	F
T	F	F
T	T	T

OR:

A	B	A OR B
F	F	F
F	T	T
T	F	T
T	T	T

NOT:

A	NOT A
F	T
T	F

XOR:

A	B	A XOR B
F	F	F
F	T	T
T	F	T
T	T	F

Where boolean values come from:

- Comparison results: age >= 18 evaluates to TRUE or FALSE
- Direct assignment: isValid ← TRUE
- Function returns: isFinished ← END_OF_FILE()

Boolean in conditions:

text

```
IF isRaining AND NOT haveUmbrella THEN
   OUTPUT "Get wet!"
ENDIF
```

De Morgan's Laws (important for simplifying conditions):

text

NOT (A AND B) = (NOT A) OR (NOT B)

NOT (A OR B) = (NOT A) AND (NOT B)

Example of De Morgan:
Original: IF NOT (age >= 18 AND hasLicense = TRUE)
Simplified: IF age < 18 OR hasLicense = FALSE

8.2.3 Type Conversion (Casting)

Implicit conversion (automatic):
The language automatically converts between compatible types.

Example:

text

```
intValue ← 5
realValue ← intValue   // 5 becomes 5.0 automatically
```

Explicit conversion (casting):
You tell the language to convert.

Syntax examples:

text

```
// String to integer
intValue ← INT("123")       // 123

// String to real
realValue ← REAL("3.14")    // 3.14

// Integer to string
strValue ← STRING(42)       // "42"

// Real to integer (truncates)
intValue ← INTEGER(7.9)     // 7
```

Dangerous conversions:

Conversion	Result	Problem
INT("hello")	Error	Cannot convert non-number
INT("12.5")	12 or error?	Loss of decimal
INT(" 123 ")	May keep spaces or error	Whitespace issues

Safe conversion pattern:

text

```
INPUT userInput
```

```
IF IS_NUMERIC(userInput) THEN
   number ← INT(userInput)
ELSE
   OUTPUT "Please enter a number"
ENDIF
```

PART 8.3: INPUT AND OUTPUT (Complete)

8.3.1 Input Operations

Purpose: Get data from the outside world into the program.

Sources of input:

- Keyboard (user typing)
- Mouse clicks
- Files on disk
- Network/sensors
- Other programs

Basic input syntax:

text

```
INPUT variableName
```

With prompt:

text

```
OUTPUT "Enter your age: "
INPUT age
```

Multiple inputs:

text

```
INPUT name, age, score
```

User would type: Zaki 25 95

Input validation pattern (critical for robust programs):

text

```
REPEAT
   OUTPUT "Enter age (1-120): "
   INPUT age
   IF age < 1 OR age > 120 THEN
      OUTPUT "Invalid. Age must be 1-120."
   ENDIF
UNTIL age >= 1 AND age <= 120
```

Reading different data types:

```
text
// String input
INPUT name

// Integer input
INPUT age
age ← INT(age)   // if input is string, convert

// Real input
INPUT height
height ← REAL(height)

// Boolean input (often yes/no)
INPUT response
isConfirmed ← (response = "yes")
```

8.3.2 Output Operations

Purpose: Display results to the user or save to files.

Basic output syntax:

```
text
OUTPUT expression
```

Outputting multiple items:

text

```
OUTPUT "Hello ", name, "! You are ", age, " years old."
```

Formatting output:

text

```
// Fixed decimal places
OUTPUT "Total: $", total TO 2 DECIMAL PLACES

// Field width (right-aligned)
OUTPUT "Score:", score WIDTH 5

// New lines
OUTPUT "First line"
OUTPUT "Second line"    // automatically adds newline

// No newline (if supported)
OUTPUT "Loading..." NO NEWLINE
OUTPUT "Done"
```

Special output characters:

text

```
OUTPUT "Column1\tColumn2\tColumn3"   // tabs
OUTPUT "Line1\nLine2\nLine3"         // newlines
```

PART 8.4: ARITHMETIC OPERATORS (Complete Reference)

8.4.1 The Six Basic Operators

Operator	Name	Example	Result
+	Addition	15 + 3	18
-	Subtraction	15 - 3	12
*	Multiplication	15 * 3	45
/	Division (real)	15 / 3	5.0
DIV	Integer division	15 DIV 4	3
MOD	Modulo (remainder)	15 MOD 4	3

8.4.2 Integer Division (DIV) Deep Dive

What it does: Divides two integers and returns the quotient (whole number part), discarding the remainder.

Examples:

Calculation	Result	Explanation
10 DIV 3	3	3 × 3 = 9, remainder 1
20 DIV 5	4	4 × 5 = 20, remainder 0
7 DIV 10	0	0 × 10 = 0, remainder 7
-10 DIV 3	-3 or -4?	Depends on language (truncates toward zero or floor)

Use cases for DIV:

1. Converting seconds to minutes: minutes = totalSeconds DIV 60
2. Finding which page an item is on: pageNumber = (itemNumber - 1) DIV itemsPerPage + 1
3. Extracting digits from right: hundreds = number DIV 100 MOD 10

8.4.3 Modulo (MOD) Deep Dive

What it does: Returns the remainder after integer division.

Relationship: a = (a DIV b) * b + (a MOD b)

Examples:

Calculation	Result	Explanation
10 MOD 3	1	3×3=9, remainder 1
20 MOD 5	0	Exactly divisible
7 MOD 10	7	0×10=0, remainder 7
10 MOD 7	3	1×7=7, remainder 3

Common uses for MOD (MEMORIZE THESE):

1. Check if a number is even or odd:

```
text
IF number MOD 2 = 0 THEN
   OUTPUT "Even"
ELSE
   OUTPUT "Odd"
ENDIF
```

2. Check if divisible by another number:

```
text
IF number MOD 7 = 0 THEN
   OUTPUT "Multiple of 7"
ENDIF
```

3. Extract last digit:

```
text
lastDigit = number MOD 10
```

Example: 123 MOD 10 = 3

4. Extract last two digits:

text

lastTwoDigits = number MOD 100

Example: 1234 MOD 100 = 34

5. Wrap around (clock arithmetic):

text

hours12 = hours24 MOD 12

IF hours12 = 0 THEN hours12 = 12

6. Cycling through array indices:

text

index = (index + 1) MOD arraySize

8.4.4 Operator Precedence (Order of Operations)

PEMDAS / BODMAS:

Level	Operators	Example
1 (highest)	Parentheses ()	(2 + 3) * 4 = 20
2	Exponentiation ^ (if available)	2^3 = 8
3	Multiplication *, Division /, DIV, MOD	2 + 3 * 4 = 14
4 (lowest)	Addition +, Subtraction -	10 - 3 + 2 = 9

Same level: Evaluate left to right.

Examples with step-by-step:

3 + 4 * 2

1. Multiplication first: 4 * 2 = 8
2. Then addition: 3 + 8 = 11

(3 + 4) * 2

1. Parentheses first: 3 + 4 = 7
2. Then multiplication: 7 * 2 = 14

20 / 4 * 2

1. Left to right: 20 / 4 = 5
2. Then: 5 * 2 = 10

20 / (4 * 2)

1. Parentheses first: 4 * 2 = 8
2. Then: 20 / 8 = 2.5

17 MOD 5 + 3 * 2

1. MOD first (same level as *): 17 MOD 5 = 2
2. Then multiplication: 3 * 2 = 6
3. Then addition: 2 + 6 = 8

8.4.5 Compound Assignment Operators (Shorthand)

These are shortcuts for common operations:

Shorthand	Long form	Example
x += 5	x = x + 5	total += 10
x -= 5	x = x - 5	remaining -= 1
x *= 5	x = x * 5	product *= factor
x /= 5	x = x / 5	average /= count
x ++	x = x + 1	count ++
x --	x = x - 1	remaining --

Note: Not all pseudocode standards include these, but most real languages do.

PART 8.5: SEQUENCE (The Simplest Control Structure)

8.5.1 What is Sequence?

Definition: Statements are executed one after another, in the order they appear in the code.

This is the default. Unless you use selection (IF) or iteration (loops), your program runs sequentially.

Example:

text

OUTPUT "Step 1"

OUTPUT "Step 2"

OUTPUT "Step 3"

Output:

text

Step 1

Step 2

Step 3

8.5.2 Flow of Execution

Visual representation:

text

[Start]

▼

[Statement 1]

▼

[Statement 2]

▼

[Statement 3]

▼

[End]

8.5.3 Why Sequence Matters

Even though sequence is simple, understanding it is critical for debugging.

Common sequence mistakes:

Mistake 1: Statements in wrong order

text

```
// Wrong: Using value before it's set
OUTPUT total   // total has no value yet!
total ← 5 + 3

// Correct:
total ← 5 + 3
OUTPUT total
```

Mistake 2: Forgetting that statements happen instantly

text

```
x ← 5
x ← x + 1   // x becomes 6 immediately
OUTPUT x    // Outputs 6, not 5
```

Mistake 3: Assuming parallel execution

text

```
x ← 5
y ← x + 2   // y becomes 7 (uses new x value)
x ← 10      // x becomes 10, but y is still 7!
```

PART 8.6: SELECTION (Making Decisions)

8.6.1 The IF Statement (Complete)

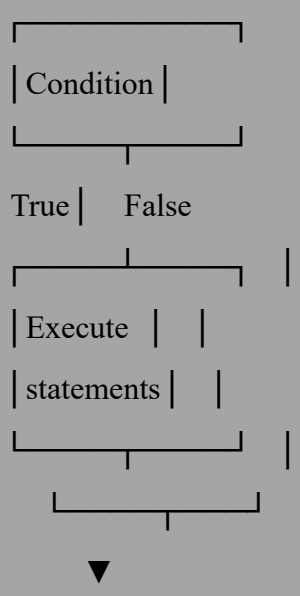
Single branch (IF-THEN):

text

```
IF condition THEN
   statements
ENDIF
```

Flowchart:

text



Example:

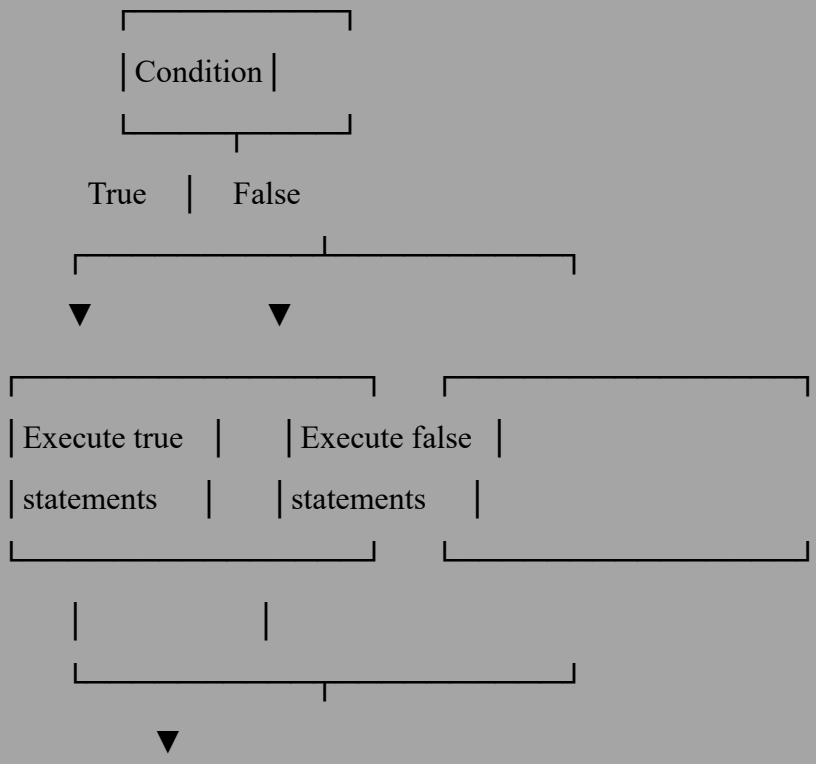
```
text
IF temperature > 30 THEN
   OUTPUT "It's hot outside!"
ENDIF
```

Two branches (IF-THEN-ELSE):

```
text
IF condition THEN
   statements_if_true
ELSE
   statements_if_false
ENDIF
```

Flowchart:

text



Example:

text

```
IF age >= 18 THEN
   OUTPUT "You can vote"
ELSE
   OUTPUT "You are too young to vote"
ENDIF
```

8.6.2 Nested IF Statements

What is nesting? Putting an IF statement inside another IF statement.

Example – Grading system:

text

```
IF mark >= 70 THEN
   grade ← "A"
ELSE
```

```
    IF mark >= 60 THEN
        grade ← "B"
    ELSE
        IF mark >= 50 THEN
            grade ← "C"
        ELSE
            grade ← "Fail"
        ENDIF
    ENDIF
ENDIF
```

Nested IF with AND condition (equivalent):

```
IF mark >= 70 THEN
    grade ← "A"
ELSE IF mark >= 60 THEN
    grade ← "B"
ELSE IF mark >= 50 THEN
    grade ← "C"
ELSE
    grade ← "Fail"
ENDIF
```

Deep nesting (caution): Too many nested IFs makes code hard to read. Consider using CASE or refactoring.

8.6.3 The CASE Statement (Multi-way Selection)

When to use: When you have one variable being compared to many specific values.

Syntax:

```
CASE expression OF
    value1: statements1
```

```
    value2: statements2
    value3: statements3
    OTHERWISE: default_statements
ENDCASE
```

Example – Day of week:

```
text
CASE dayNumber OF
    1: OUTPUT "Monday"
    2: OUTPUT "Tuesday"
    3: OUTPUT "Wednesday"
    4: OUTPUT "Thursday"
    5: OUTPUT "Friday"
    6: OUTPUT "Saturday"
    7: OUTPUT "Sunday"
    OTHERWISE: OUTPUT "Invalid day number"
ENDCASE
```

Example – Menu system:

```
text
OUTPUT "1. Add"
OUTPUT "2. Subtract"
OUTPUT "3. Multiply"
OUTPUT "4. Divide"
OUTPUT "5. Exit"
INPUT choice

CASE choice OF
    1: CALL addNumbers()
    2: CALL subtractNumbers()
```

```
    3: CALL multiplyNumbers()
    4: CALL divideNumbers()
    5: OUTPUT "Goodbye"
    OTHERWISE: OUTPUT "Invalid choice"
ENDCASE
```

CASE vs IF-ELSE IF:

Aspect	CASE	IF-ELSE IF
Readability	Better for many equal comparisons	Better for ranges
Performance	Can be faster (jump table)	Evaluates each condition
Flexibility	Only equality checks	Any condition
When to use	Menu options, state machines	Ranges, complex conditions

8.6.4 Conditional Expressions (Ternary Operator)

What it is: A shorthand for simple IF-ELSE that returns a value.

Syntax: result ← IF condition THEN value_if_true ELSE value_if_false

Example:

```
text
// Long form
IF age >= 18 THEN
    status ← "Adult"
ELSE
    status ← "Minor"
ENDIF

// Short form (ternary)
status ← IF age >= 18 THEN "Adult" ELSE "Minor"
```

Use with care: Ternary is concise but can be hard to read if overused.

8.6.5 Common Selection Patterns

Pattern 1: Validation (check then process)

```
text
IF age >= 1 AND age <= 120 THEN
   // Process valid age
   OUTPUT "Age accepted"
ELSE
   OUTPUT "Invalid age"
ENDIF
```

Pattern 2: Guard clause (early exit)

```
text
IF age < 18 THEN
   OUTPUT "Too young"
   RETURN   // Exit the subroutine
ENDIF
// Continue with adult processing
```

Pattern 3: Multiple independent checks

```
text
IF score >= 90 THEN OUTPUT "A"
IF score >= 80 AND score < 90 THEN OUTPUT "B"
IF score >= 70 AND score < 80 THEN OUTPUT "C"
```

(Note: These are independent; all conditions checked)

Pattern 4: Exclusive ranges (use ELSE IF)

```
text
IF score >= 90 THEN
   OUTPUT "A"
ELSE IF score >= 80 THEN
```

```
    OUTPUT "B"
ELSE IF score >= 70 THEN
    OUTPUT "C"
ELSE
    OUTPUT "Fail"
ENDIF
```

(Only one branch executes)

PART 8.7: ITERATION (Loops) – Complete

8.7.1 Why Loops?

Problem without loops: Repeating code manually.

```
text
OUTPUT 1
OUTPUT 2
OUTPUT 3
... (1000 lines of OUTPUT)
```

Solution with loops:

```
text
FOR i ← 1 TO 1000
    OUTPUT i
NEXT i
```

Loops are for repetition – doing the same thing many times.

8.7.2 FOR Loop (Count-Controlled)

When to use: You know exactly how many times to repeat.

Syntax:

```
text
FOR counter ← start TO end [STEP increment]
    statements
```

NEXT counter

Basic example:

text

```
FOR i ← 1 TO 5
    OUTPUT "Hello"
NEXT i
```

Outputs "Hello" 5 times.

Using the counter:

text

```
FOR i ← 1 TO 5
    OUTPUT "Number: ", i
NEXT i
```

Output:

text

```
Number: 1
Number: 2
Number: 3
Number: 4
Number: 5
```

Counting backwards:

text

```
FOR i ← 10 DOWNTO 1
    OUTPUT i
NEXT i
```

Output: 10, 9, 8, 7, 6, 5, 4, 3, 2, 1

Using STEP:

text

```
FOR i ← 1 TO 10 STEP 2
```

```
    OUTPUT i
NEXT i
```

Output: 1, 3, 5, 7, 9

STEP can be negative:

text

```
FOR i ← 10 DOWNTO 1 STEP 2
    OUTPUT i
NEXT i
```

Output: 10, 8, 6, 4, 2

Common FOR loop patterns:

Pattern	Code	Result
Sum 1 to n	total ← 0 FOR i ← 1 TO n total ← total + i NEXT i	total = n(n+1)/2
Factorial	fact ← 1 FOR i ← 1 TO n fact ← fact * i NEXT i	fact = n!
Array traversal	FOR i ← 0 TO size-1 OUTPUT arr[i] NEXT i	Print all elements
Reverse array	FOR i ← 0 TO size/2 - 1 swap arr[i] and arr[size-1-i] NEXT i	Reverse order

8.7.3 WHILE Loop (Condition-Controlled, Pre-Test)

When to use: You don't know how many times, but you have a condition to continue.

Syntax:

text

```
WHILE condition DO
   statements
ENDWHILE
```

Important: The condition is checked BEFORE each iteration. If initially false, loop runs 0 times.

Example – Count to 5:

text

```
i ← 1
WHILE i <= 5 DO
   OUTPUT i
   i ← i + 1
ENDWHILE
```

Example – Keep asking until valid:

text

```
INPUT age
WHILE age < 0 OR age > 120 DO
   OUTPUT "Invalid. Enter age 1-120: "
   INPUT age
ENDWHILE
```

Example – Sum unknown number of inputs (sentinel value):

text

```
total ← 0
INPUT number
WHILE number != -1 DO    // -1 is the sentinel (stop marker)
   total ← total + number
   INPUT number
ENDWHILE
OUTPUT "Total: ", total
```

Example – Menu system:

```
text
choice ← 0
WHILE choice != 5 DO
   OUTPUT "1. Option 1"
   OUTPUT "2. Option 2"
   OUTPUT "3. Option 3"
   OUTPUT "4. Option 4"
   OUTPUT "5. Exit"
   INPUT choice

   IF choice = 1 THEN CALL option1()
   IF choice = 2 THEN CALL option2()
   IF choice = 3 THEN CALL option3()
   IF choice = 4 THEN CALL option4()
ENDWHILE
```

8.7.4 REPEAT UNTIL Loop (Condition-Controlled, Post-Test)

When to use: You need to run the loop at least once before checking.

Syntax:

```
text
REPEAT
   statements
UNTIL condition
```

Important: The condition is checked AFTER each iteration. The loop stops when condition becomes TRUE. Runs at least once.

Example – Count to 5:

```
text
i ← 1
REPEAT
```

```
    OUTPUT i
    i ← i + 1
UNTIL i > 5
```

Example – Menu system (runs at least once):

```
text
REPEAT
    OUTPUT "1. Add"
    OUTPUT "2. Subtract"
    OUTPUT "3. Exit"
    INPUT choice

    IF choice = 1 THEN CALL add()
    IF choice = 2 THEN CALL subtract()
UNTIL choice = 3
```

Example – Input validation (asks at least once):

```
text
REPEAT
    OUTPUT "Enter age (1-120): "
    INPUT age
    IF age < 1 OR age > 120 THEN
        OUTPUT "Invalid"
    ENDIF
UNTIL age >= 1 AND age <= 120
```

8.7.5 WHILE vs REPEAT UNTIL – Critical Differences

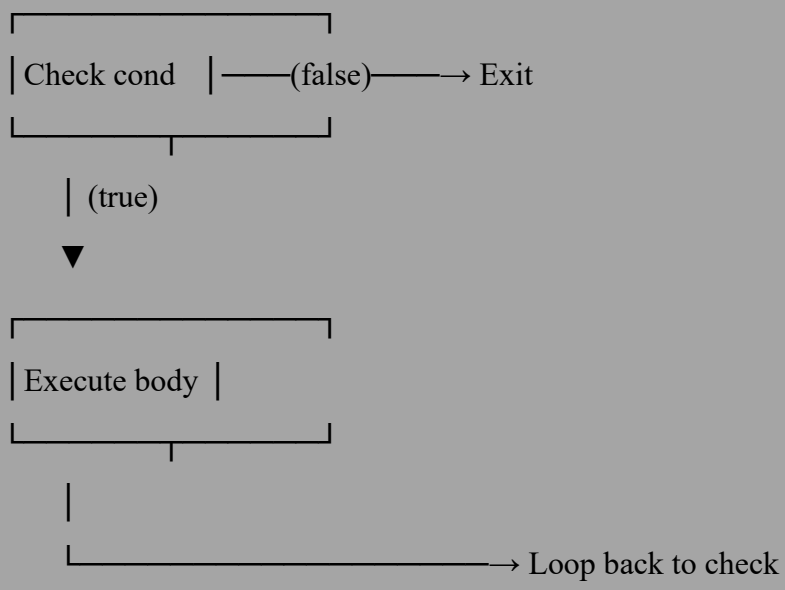
Aspect	WHILE	REPEAT UNTIL
Check timing	Before loop (pre-test)	After loop (post-test)

Aspect	WHILE	REPEAT UNTIL
Minimum runs	0 (may not run)	1 (always runs at least once)
Condition meaning	Continue while true	Stop when true
Typical use	Validation, file reading	Menu, "do once then check"

Visual comparison:

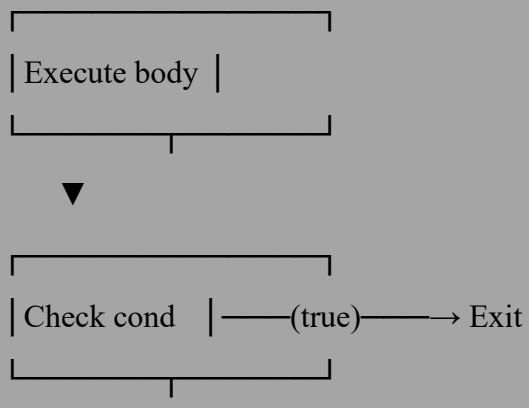
WHILE:

text



REPEAT UNTIL:

text



Choosing which to use:

- Use **FOR** when you know the exact count
- Use **WHILE** when you might need to run 0 times (e.g., checking before processing)
- Use **REPEAT UNTIL** when you must run at least once (e.g., displaying a menu)

8.7.6 Infinite Loops and How to Avoid Them

What is an infinite loop? A loop that never ends.

Common causes:

1. Condition never becomes false
2. Forgetting to update loop variable
3. Wrong comparison operator

Examples of infinite loops:

Cause 1 – Condition always true:

text

```
i ← 1
WHILE i <= 5 DO
   OUTPUT i
   // Missing: i ← i + 1
ENDWHILE
```

i never changes, so i <= 5 is always true.

Cause 2 – Wrong operator:

text

```
i ← 1
WHILE i != 5 DO
   OUTPUT i
   i ← i + 2
ENDWHILE
```

i becomes 1,3,5,7,9,... never equals 5 (skips over it).

Cause 3 – Logic error:

text

```
WHILE score >= 0 DO
   INPUT score   // If user enters only positive numbers...
ENDWHILE
```

How to prevent:

1. Always update loop variables inside the loop
2. Ensure condition can eventually become false
3. Add a maximum iteration safety check (if language supports)
4. Test with boundary values

8.7.7 Nested Loops (Loops Inside Loops)

What are nested loops? A loop inside another loop.

Inner loop runs completely for each iteration of outer loop.

Example – Multiplication table:

text

```
FOR i ← 1 TO 10
   FOR j ← 1 TO 10
      OUTPUT i * j
   NEXT j
   OUTPUT newline   // New line after each row
NEXT i
```

How nested loops execute:

text

```
Outer i=1:
   Inner j=1: output 1
   Inner j=2: output 2
   ...
```

Inner j=10: output 10

(inner loop finishes)

Outer i=2:

Inner j=1: output 2

Inner j=2: output 4

...

Inner j=10: output 20

...

Outer i=10:

Inner runs 10 times

Total iterations: Outer × Inner = 10 × 10 = 100 outputs.

Example – Star pattern:

text

```
FOR i ← 1 TO 5
    FOR j ← 1 TO i
        OUTPUT "*"
    NEXT j
    OUTPUT newline
NEXT i
```

Output:

text

```
*
**
***
****
*****
```

Example – Clock simulation:

text

```
FOR hour ← 0 TO 23
   FOR minute ← 0 TO 59
      FOR second ← 0 TO 59
         OUTPUT hour, ":", minute, ":", second
      NEXT second
   NEXT minute
NEXT hour
```

Total iterations: 24 × 60 × 60 = 86,400 outputs.

Performance consideration: Nested loops can become very slow. An $O(n^2)$ algorithm (nested loops over same array) with n=10,000 takes 100 million iterations.

PART 8.8: TOTALING (Accumulation)

8.8.1 What is Totalling?

Definition: Keeping a running sum of values as you process them.

The accumulator pattern:

```
total ← 0          // Initialize to zero
FOR each value
   total ← total + value   // Add each value
NEXT
```

Why initialize to zero? If you don't, total starts with garbage (whatever was in memory), and your sum will be wrong.

8.8.2 Basic Totalling Examples

Example 1 – Sum of 5 input numbers:

```
total ← 0
FOR i ← 1 TO 5
   INPUT num
   total ← total + num
```

```
NEXT i
OUTPUT "Sum: ", total
```

Example 2 – Sum of numbers 1 to n:

```
text
INPUT n
total ← 0
FOR i ← 1 TO n
   total ← total + i
NEXT i
OUTPUT total
```

Example 3 – Sum of array elements:

```
text
total ← 0
FOR i ← 0 TO size-1
   total ← total + arr[i]
NEXT i
```

Example 4 – Conditional sum (sum only positive numbers):

```
text
total ← 0
FOR i ← 1 TO 10
   INPUT num
   IF num > 0 THEN
      total ← total + num
   ENDIF
NEXT i
```

8.8.3 Running Total vs Grand Total

Running total: Updated continuously as data comes in.
Grand total: The final sum after all data is processed.

Example – Cash register:

text

```
runningTotal ← 0
REPEAT
   INPUT price
   IF price = 0 THEN EXIT
   runningTotal ← runningTotal + price
   OUTPUT "Current total: $", runningTotal
UNTIL price = 0
OUTPUT "Grand total: $", runningTotal
```

8.8.4 Weighted Sum

What is weighted sum? Each value is multiplied by a weight before adding.

Example – Grade calculation with weighted categories:

text

```
// Homework (40% of grade)
homeworkAvg ← 85
// Exams (60% of grade)
examAvg ← 75

finalGrade ← (homeworkAvg * 0.4) + (examAvg * 0.6)
// finalGrade = 85*0.4 + 75*0.6 = 34 + 45 = 79
```

Example – Moving average (simplified):

text

```
sum ← 0
FOR i ← 1 TO 5
   sum ← sum + values[i]
NEXT i
movingAvg ← sum / 5   // Average of last 5 values
```

PART 8.9: COUNTING

8.9.1 What is Counting?

Definition: Keeping track of how many times something occurs.

The counter pattern:

text

```
counter ← 0          // Initialize to zero
FOR each item
   IF condition THEN
      counter ← counter + 1   // Increment when condition met
   ENDIF
NEXT
```

8.9.2 Basic Counting Examples

Example 1 – Count even numbers:

text

```
evenCount ← 0
FOR i ← 1 TO 10
   INPUT num
   IF num MOD 2 = 0 THEN
      evenCount ← evenCount + 1
   ENDIF
NEXT i
OUTPUT "Even numbers: ", evenCount
```

Example 2 – Count passing scores:

text

```
passCount ← 0
FOR i ← 1 TO 20
   INPUT score
   IF score >= 50 THEN
```

```
        passCount ← passCount + 1
    ENDIF
NEXT i
OUTPUT "Passed: ", passCount
```

Example 3 – Count vowels in a string:

```
text
vowelCount ← 0
FOR i ← 1 TO LENGTH(text)
    char ← SUBSTRING(text, i, 1)
    char ← LOWER(char)
    IF char = "a" OR char = "e" OR char = "i" OR char = "o" OR char = "u" THEN
        vowelCount ← vowelCount + 1
    ENDIF
NEXT i
```

8.9.3 Frequency Counting (Histogram)

What is frequency counting? Counting how many times each distinct value appears.

Example – Dice rolls (1-6):

```
text
DECLARE frequency[6] : INTEGER
FOR i ← 0 TO 5
    frequency[i] ← 0
NEXT i

FOR roll ← 1 TO 100
    result ← RANDOM(1, 6)
    frequency[result-1] ← frequency[result-1] + 1
NEXT roll
```

```
FOR i ← 0 TO 5
    OUTPUT "Number ", i+1, " appeared ", frequency[i], " times"
NEXT i
```

8.9.4 Counter vs Accumulator

	Counter	Accumulator (Total)
What it stores	Number of occurrences	Sum of values
Initial value	0	0
Operation	Add 1	Add value
Final meaning	Count	Sum

Combined example – Average calculation:

```
text
sum ← 0          // accumulator
count ← 0        // counter

FOR each value
    sum ← sum + value
    count ← count + 1
NEXT

IF count > 0 THEN
    average ← sum / count
ENDIF
```

PART 8.10: STRING MANIPULATION (Complete)

8.10.1 String Fundamentals (Review)

String: A sequence of characters.

Empty string: "" (zero characters)

String length: Number of characters.

Indexing: Accessing individual characters.

8.10.2 Complete String Function Reference

Assuming **1-based indexing** (common in pseudocode):

Function	Syntax	Example	Result
Length	LENGTH(str)	LENGTH("Hello")	5
Substring	SUBSTRING(str, start, count)	SUBSTRING("Hello",2,3)	"ell"
Left	LEFT(str, n)	LEFT("Hello",2)	"He"
Right	RIGHT(str, n)	RIGHT("Hello",2)	"lo"
Mid (alternative)	MID(str, start, count)	MID("Hello",2,3)	"ell"
Position	POSITION(sub, str)	POSITION("ll","Hello")	3
Uppercase	UPPER(str)	UPPER("Hello")	"HELLO"
Lowercase	LOWER(str)	LOWER("Hello")	"hello"
Trim (remove spaces)	TRIM(str)	TRIM(" Hi ")	"Hi"
Character at index	str[index]	"Hello"[3]	'l'

8.10.3 String Manipulation Examples

Example 1 – Extract first name from full name:

text

```
fullName ← "Zaki Ahmed"
spacePos ← POSITION(" ", fullName)
firstName ← LEFT(fullName, spacePos - 1)
OUTPUT firstName   // "Zaki"
```

Example 2 – Extract last name:

```
text
fullName ← "Zaki Ahmed"
spacePos ← POSITION(" ", fullName)
lastName ← RIGHT(fullName, LENGTH(fullName) - spacePos)
OUTPUT lastName   // "Ahmed"
```

Example 3 – Reverse a string:

```
text
INPUT str
reversed ← ""
FOR i ← LENGTH(str) DOWNTO 1
   reversed ← reversed + SUBSTRING(str, i, 1)
NEXT i
OUTPUT reversed
```

Example 4 – Check if string is palindrome:

```
text
INPUT str
isPalindrome ← TRUE
FOR i ← 1 TO LENGTH(str) DIV 2
   IF SUBSTRING(str, i, 1) <> SUBSTRING(str, LENGTH(str)-i+1, 1) THEN
      isPalindrome ← FALSE
      EXIT
   ENDIF
NEXT i
```

```
IF isPalindrome THEN
   OUTPUT "Palindrome"
ELSE
   OUTPUT "Not palindrome"
ENDIF
```

Example 5 – Convert to title case (first letter uppercase, rest lowercase):

```
text
INPUT str
str ← LOWER(str)
firstChar ← UPPER(LEFT(str, 1))
rest ← RIGHT(str, LENGTH(str)-1)
result ← firstChar + rest
OUTPUT result
```

Example 6 – Count words:

```
text
INPUT sentence
wordCount ← 1   // Assume at least one word
FOR i ← 1 TO LENGTH(sentence)
   IF SUBSTRING(sentence, i, 1) = " " THEN
      wordCount ← wordCount + 1
   ENDIF
NEXT i
OUTPUT "Words: ", wordCount
```

Example 7 – Replace all occurrences:

```
text
INPUT text, oldSub, newSub
result ← ""
```

```
i ← 1
WHILE i <= LENGTH(text)
   IF SUBSTRING(text, i, LENGTH(oldSub)) = oldSub THEN
      result ← result + newSub
      i ← i + LENGTH(oldSub)
   ELSE
      result ← result + SUBSTRING(text, i, 1)
      i ← i + 1
   ENDIF
ENDWHILE
OUTPUT result
```

8.10.4 String Comparison Rules

How strings are compared:

1. Compare first characters (using ASCII/Unicode values)
2. If different, that determines the result
3. If same, move to next character
4. If all characters match but one string is longer, the shorter is "less"

Examples:

Comparison	Result	Why
"Apple" < "Banana"	TRUE	'A'(65) < 'B'(66)
"apple" < "banana"	TRUE	'a'(97) < 'b'(98)
"Apple" < "apple"	TRUE	'A'(65) < 'a'(97)
"Hi" < "Hi!"	TRUE	Same prefix, "Hi" is shorter
"Hello" = "hello"	FALSE	'H'(72) vs 'h'(104)

Case-insensitive comparison:

text

```
IF UPPER(str1) = UPPER(str2) THEN
   OUTPUT "Same (ignoring case)"
ENDIF
```

PART 8.11: NESTED STATEMENTS (Complex Control Flow)

8.11.1 What are Nested Statements?

Definition: Putting one control structure (IF, loop) inside another.

Why nest? To handle complex logic that requires multiple levels of decision-making.

8.11.2 Nested IF Examples

Example 1 – Three-level decision:

text

```
IF age >= 18 THEN
   IF hasLicense THEN
      IF passedTest THEN
         OUTPUT "Can drive"
      ELSE
         OUTPUT "Need to pass test"
      ENDIF
   ELSE
      OUTPUT "Need license"
   ENDIF
ELSE
   OUTPUT "Too young"
ENDIF
```

Example 2 – Grade with +/-:

text

```
IF mark >= 70 THEN
   grade ← "A"
   IF mark >= 90 THEN
      grade ← grade + "+"
   ELSE IF mark <= 75 THEN
      grade ← grade + "-"
   ENDIF
ELSE IF mark >= 60 THEN
   grade ← "B"
   // similar logic
ENDIF
```

8.11.3 Nested Loop Examples

Example 1 – Multiplication table (already seen):

```
text
FOR i ← 1 TO 10
   FOR j ← 1 TO 10
      OUTPUT i * j
   NEXT j
   OUTPUT newline
NEXT i
```

Example 2 – Find duplicate pairs:

```
text
FOR i ← 0 TO size-2
   FOR j ← i+1 TO size-1
      IF arr[i] = arr[j] THEN
         OUTPUT "Duplicate found: ", arr[i]
      ENDIF
   NEXT j
```

```
NEXT i
```

Example 3 – Bubble sort (nested loops):

```
text
FOR i ← 0 TO n-2
   FOR j ← 0 TO n-i-2
      IF arr[j] > arr[j+1] THEN
         swap arr[j] and arr[j+1]
      ENDIF
   NEXT j
NEXT i
```

8.11.4 Mixed Nesting (IF inside Loop, Loop inside IF)

IF inside loop:

```
text
FOR i ← 1 TO 100
   IF i MOD 2 = 0 THEN
      OUTPUT i, " is even"
   ELSE
      OUTPUT i, " is odd"
   ENDIF
NEXT i
```

Loop inside IF:

```
text
IF userChoice = "sum" THEN
   total ← 0
   FOR i ← 1 TO 10
      total ← total + numbers[i]
   NEXT i
   OUTPUT total
```

```
ENDIF
```

8.11.5 Deep Nesting and Readability

Problem with deep nesting:

```
text
IF condition1 THEN
   IF condition2 THEN
      IF condition3 THEN
         IF condition4 THEN
            // Hard to track which IF this belongs to
         ENDIF
      ENDIF
   ENDIF
ENDIF
```

Solutions:

1. **Refactor into subroutines**
2. **Combine conditions** (AND/OR)
3. **Use early returns** (guard clauses)

Refactored example:

```
text
PROCEDURE validateAll()
   IF NOT condition1 THEN RETURN
   IF NOT condition2 THEN RETURN
   IF NOT condition3 THEN RETURN
   IF NOT condition4 THEN RETURN
   // All conditions passed
ENDPROCEDURE
```

PART 8.12: SUBROUTINES (Procedures and Functions)

8.12.1 What are Subroutines?

Definition: A named block of code that performs a specific task and can be called (executed) from anywhere in the program.

Benefits:

- **Modularity** – Break large programs into smaller pieces
- **Reusability** – Write once, use many times
- **Testability** – Test each subroutine independently
- **Readability** – Main program becomes high-level overview

8.12.2 Procedures vs Functions

Aspect	Procedure	Function
Purpose	Perform an action	Calculate and return a value
Returns	Nothing	One value (of specific type)
Called as	CALL name(args)	variable ← name(args)
Example	printGreeting("Zaki")	result ← double(5)
Side effects	Often has (e.g., output)	Should be minimal

8.12.3 Procedure (Complete)

Syntax:

text

```
PROCEDURE procedureName(parameters)
    statements
ENDPROCEDURE
```

Calling a procedure:

text

```
CALL procedureName(arguments)
```

Example – Simple procedure:

text

```
PROCEDURE sayHello()
   OUTPUT "Hello!"
ENDPROCEDURE

CALL sayHello()   // Outputs: Hello!
```

Example – Procedure with parameters:

text

```
PROCEDURE greet(name)
   OUTPUT "Hello, ", name
ENDPROCEDURE

CALL greet("Zaki")   // Outputs: Hello, Zaki
```

Example – Procedure with multiple parameters:

text

```
PROCEDURE printSum(a, b)
   sum ← a + b
   OUTPUT a, " + ", b, " = ", sum
ENDPROCEDURE

CALL printSum(5, 3)   // Outputs: 5 + 3 = 8
```

8.12.4 Function (Complete)

Syntax:

text

```
FUNCTION functionName(parameters) RETURNS dataType
   statements
   RETURN value
ENDFUNCTION
```

Calling a function:

text

```
result ← functionName(arguments)
```

Example – Simple function:

text

```
FUNCTION double(x) RETURNS INTEGER
   RETURN x * 2
ENDFUNCTION

result ← double(5)   // result = 10
```

Example – Function with multiple parameters:

text

```
FUNCTION add(a, b) RETURNS INTEGER
   RETURN a + b
ENDFUNCTION

sum ← add(5, 3)   // sum = 8
```

Example – Function with logic:

text

```
FUNCTION getGrade(mark) RETURNS STRING
   IF mark >= 70 THEN
      RETURN "A"
   ELSE IF mark >= 60 THEN
      RETURN "B"
   ELSE IF mark >= 50 THEN
      RETURN "C"
   ELSE
      RETURN "Fail"
```

```
    ENDIF
ENDFUNCTION


grade ← getGrade(75)   // grade = "A"
```

8.12.5 Parameters and Arguments (Detailed)

Parameter: The variable name in the subroutine definition.
Argument: The actual value passed when calling.

Example:

```
text
PROCEDURE greet(name)   // 'name' is a parameter
    OUTPUT "Hello, ", name
ENDPROCEDURE


CALL greet("Zaki")      // "Zaki" is an argument
```

8.12.6 Pass by Value vs Pass by Reference

Pass by Value (default, usually):

- A copy of the argument is made
- Changes inside subroutine do NOT affect original

Example – Pass by value:

```
text
PROCEDURE tryToChange(x)
    x ← 100
ENDPROCEDURE


num ← 5
CALL tryToChange(num)
OUTPUT num   // Still 5 (unchanged!)
```

Pass by Reference (explicit with BYREF):

- The original variable is used

- Changes inside subroutine DO affect original

Example – Pass by reference:

```
text
PROCEDURE actuallyChange(BYREF x)
   x ← 100
ENDPROCEDURE

num ← 5
CALL actuallyChange(num)
OUTPUT num   // Now 100 (changed!)
```

When to use each:

Use pass by value	Use pass by reference
When you don't want to modify original	When you need to modify original
For simple calculations	For swapping values
Default choice	When multiple values need to be returned

8.12.7 Variable Scope in Subroutines (Review)

Local variables: Declared inside subroutine, only accessible there.

Global variables: Declared outside all subroutines, accessible everywhere.

Example:

```
text
DECLARE globalVar : INTEGER   // Global

PROCEDURE test()
   DECLARE localVar : INTEGER   // Local
   globalVar ← 10   // OK
   localVar ← 20    // OK
```

```
ENDPROCEDURE


// localVar cannot be accessed here (out of scope)
globalVar ← 5   // OK
```

8.12.8 Recursion (Advanced)

What is recursion? A function that calls itself.

Two requirements:

1. Base case (stopping condition)
2. Recursive case (calls itself with smaller problem)

Example – Factorial recursively:

text

```
FUNCTION factorial(n) RETURNS INTEGER
   IF n = 0 OR n = 1 THEN
      RETURN 1          // Base case
   ELSE
      RETURN n * factorial(n - 1)   // Recursive case
   ENDIF
ENDFUNCTION
```

How factorial(5) executes:

text

```
factorial(5) = 5 * factorial(4)
             = 5 * (4 * factorial(3))
             = 5 * (4 * (3 * factorial(2)))
             = 5 * (4 * (3 * (2 * factorial(1))))
             = 5 * (4 * (3 * (2 * 1)))
             = 5 * (4 * (3 * 2))
             = 5 * (4 * 6)
             = 5 * 24
```

= 120

Example – Fibonacci recursively:

```
text
FUNCTION fib(n) RETURNS INTEGER
   IF n = 0 THEN
      RETURN 0
   ELSE IF n = 1 THEN
      RETURN 1
   ELSE
      RETURN fib(n-1) + fib(n-2)
   ENDIF
ENDFUNCTION
```

Recursion vs Iteration:

Aspect	Recursion	Iteration (Loop)
Readability	Often cleaner	Can be messy
Performance	Slower (function calls)	Faster
Memory	Uses stack (risk of overflow)	Less memory
When to use	Tree traversal, divide & conquer	Most other cases

---

PART 8.13: LIBRARY ROUTINES

8.13.1 What are Library Routines?

Definition: Pre-written, tested, reusable code provided by the programming language.

Benefits:

- Don't reinvent the wheel
- Usually optimized for performance

- Already debugged
- Standardized across programs

8.13.2 Common Library Categories

1. Math Library:

Function	Description	Example
SQRT(x)	Square root	SQRT(25) = 5
POW(x,y)	x to the power y	POW(2,3) = 8
ABS(x)	Absolute value	ABS(-5) = 5
ROUND(x)	Round to nearest integer	ROUND(3.6) = 4
FLOOR(x)	Round down	FLOOR(3.9) = 3
CEIL(x)	Round up	CEIL(3.1) = 4
RANDOM()	Random number	RANDOM(1,6) (dice roll)
MAX(a,b)	Larger of two	MAX(5,3) = 5
MIN(a,b)	Smaller of two	MIN(5,3) = 3

2. String Library (covered in 8.10):
LENGTH, SUBSTRING, UPPER, LOWER, POSITION, etc.

3. Date/Time Library:

Function	Description
NOW()	Current date and time
DATE()	Current date only

Function	Description
TIME()	Current time only
DAY(date)	Extract day
MONTH(date)	Extract month
YEAR(date)	Extract year

4. File Library (covered in 8.16):
OPEN, READ, WRITE, CLOSE, EOF

8.13.3 Using Library Routines

Example – Math:

text

```
radius ← 5
area ← PI * POW(radius, 2)
rounded ← ROUND(area)
```

Example – Random numbers:

text

```
// Roll a dice (1-6)
dice ← RANDOM(1, 6)

// Random decimal between 0 and 1
r ← RANDOM()

// Random integer in range
lower ← 10
upper ← 20
randomInt ← RANDOM(lower, upper + 1)
```

Example – Date calculations:

```
text
today ← DATE()
birthday ← DATE(1990, 5, 15)
age ← YEAR(today) - YEAR(birthday)
IF MONTH(today) < MONTH(birthday) THEN
   age ← age - 1
ENDIF
```

PART 8.14: MAINTAINABLE PROGRAMS

8.14.1 What is Maintainability?

Definition: How easy it is to understand, fix, and extend a program.

Why it matters:

- Most software cost is maintenance (60-80%)
- You will read code more often than you write it
- Other people (or future you) need to understand your code

8.14.2 Characteristics of Maintainable Code

Characteristic	What it means	Bad example	Good example
Readability	Easy to read and understand	x = y * 3.14	circumference = diameter * PI
Modularity	Broken into small subroutines	1000-line main program	Multiple 10-20 line functions
No duplication	Don't repeat yourself (DRY)	Same calculation 3 times	One function called 3 times
Consistent style	Follows conventions	Mixed camelCase and snake_case	Always camelCase

Characteristic	What it means	Bad example	Good example
Commented	Explains why, not what	x = x + 1 // add 1	# Increment counter for next iteration
Error handling	Deals with bad inputs	Crashes on invalid input	Validation, error messages

8.14.3 Naming Conventions (Best Practices)

Variables: Use meaningful names

Bad	Good
a, b, c	width, height, area
temp (vague)	swapTemp
flag	isComplete, hasError

Constants: UPPER_SNAKE_CASE

text

```
CONSTANT MAX_STUDENTS ← 100
CONSTANT TAX_RATE ← 0.2
```

Procedures: VerbNoun (action then object)

Bad	Good
calculate()	calculateAverage()
print()	printReceipt()
get()	getUserInput()

Functions: Noun or verb describing return value

Bad	Good
do()	double()
check()	isValid()
compute()	computeTotal()

Booleans: Use is, has, can prefix

Bad	Good
valid	isValid
finished	isFinished
license	hasLicense

8.14.4 Commenting Guidelines

Good comments explain WHY, not WHAT:

text

```
// BAD (states the obvious)
x = x + 1   // Add 1 to x

// GOOD (explains purpose)
x = x + 1   // Move to next array position
```

Commenting levels:

1. **File header:** What the whole program does
2. **Function header:** What the function does, parameters, return value
3. **Complex logic:** Explanation of non-obvious algorithms
4. **TODOs:** Notes for future improvements

Function header template:

text

```
// Calculates the average of an array of numbers
// Parameters:
//   arr - array of REAL values
//   size - number of elements in array
// Returns:
//   REAL average, or 0 if array is empty
FUNCTION calculateAverage(arr, size)
```

8.14.5 Indentation and Whitespace

Why indent: Shows structure (which statements belong inside which blocks)

Example – Bad indentation:

text

```
IF x > 5 THEN
OUTPUT "Big"
ELSE
OUTPUT "Small"
ENDIF
```

Example – Good indentation:

text

```
IF x > 5 THEN
   OUTPUT "Big"
ELSE
   OUTPUT "Small"
ENDIF
```

Indentation rules:

- Inside IF: indent 4 spaces or 1 tab
- Inside loops: indent
- Inside subroutines: indent

- Be consistent

Whitespace for readability:

text

```
// Bad
x=5;y=10;total=x+y


// Good
x ← 5
y ← 10
total ← x + y
```

8.14.6 Magic Numbers

What is a magic number? A literal value in code with no explanation.

Bad – Magic numbers:

text

```
if age > 18 {
   discount = price * 0.9   // What is 0.9?
}
```

Good – Named constants:

text

```
CONSTANT ADULT_AGE ← 18
CONSTANT STUDENT_DISCOUNT ← 0.9


IF age >= ADULT_AGE THEN
   discount ← price * STUDENT_DISCOUNT
ENDIF
```

Benefits:

- Change in one place, not everywhere
- Self-documenting code

- Easier to understand

PART 8.15: ARRAYS (Complete)

8.15.1 What is an Array?

Definition: A collection of variables of the same data type, stored in contiguous memory locations, accessed by an index.

Analogy: An array is like a row of lockers. Each locker has a number (index) and contains an item (value).

Visual representation:

text

Array: scores = [78, 84, 92, 67, 89]

Index: 0 1 2 3 4

Value: 78 84 92 67 89

8.15.2 Declaring and Using Arrays

Declaration syntax:

text

DECLARE arrayName[size] : dataType

Examples:

text

DECLARE scores[5] : INTEGER // Array of 5 integers

DECLARE names[100] : STRING // Array of 100 strings

DECLARE grid[10][20] : INTEGER // 2D array (10 rows, 20 columns)

Accessing elements:

text

scores[0] ← 78 // Set first element

scores[1] ← 84 // Set second element

OUTPUT scores[0] // Output first element

Important: Index typically starts at 0 (not 1). An array of size 5 has indices 0,1,2,3,4.

8.15.3 Common Array Operations

1. Filling an array with user input:

```
DECLARE numbers[10] : INTEGER
FOR i ← 0 TO 9
   INPUT numbers[i]
NEXT i
```

2. Displaying all elements:

```
FOR i ← 0 TO 9
   OUTPUT numbers[i]
NEXT i
```

3. Finding maximum value:

```
max ← numbers[0]
FOR i ← 1 TO 9
   IF numbers[i] > max THEN
      max ← numbers[i]
   ENDIF
NEXT i
```

4. Finding minimum value:

```
min ← numbers[0]
FOR i ← 1 TO 9
   IF numbers[i] < min THEN
      min ← numbers[i]
   ENDIF
NEXT i
```

5. Calculating sum:

text

```
sum ← 0
FOR i ← 0 TO 9
   sum ← sum + numbers[i]
NEXT i
```

6. Linear search:

text

```
FUNCTION search(arr, size, target)
   FOR i ← 0 TO size-1
      IF arr[i] = target THEN
         RETURN i
      ENDIF
   NEXT i
   RETURN -1
ENDFUNCTION
```

8.15.4 Parallel Arrays

What are parallel arrays? Two or more arrays where elements at the same index are related.

Example – Student data:

text

```
DECLARE studentNames[30] : STRING
DECLARE studentScores[30] : INTEGER

studentNames[0] ← "Zaki"
studentScores[0] ← 85

studentNames[1] ← "Ahmed"
studentScores[1] ← 92
```

```
// Output all students
FOR i ← 0 TO 29
    OUTPUT studentNames[i], " scored ", studentScores[i]
NEXT i
```

When to use: When you need to store multiple attributes and your language doesn't have records/structs.

8.15.5 2D Arrays (Tables)

What is a 2D array? An array of arrays – a grid with rows and columns.

Declaration:

text

```
DECLARE matrix[3][4] : INTEGER   // 3 rows, 4 columns
```

Visual representation:

text

```
        Column 0  Column 1  Column 2  Column 3
Row 0:   [  0   ][  1   ][  2   ][  3   ]
Row 1:   [  4   ][  5   ][  6   ][  7   ]
Row 2:   [  8   ][  9   ][ 10   ][ 11   ]
```

Accessing elements:

text

```
matrix[0][0] ← 0     // First row, first column
matrix[2][3] ← 11    // Third row, fourth column
```

Nested loops for 2D arrays:

text

```
// Fill with values
FOR row ← 0 TO 2
    FOR col ← 0 TO 3
        matrix[row][col] ← row * 4 + col
    NEXT col
NEXT row
```

```
// Display as grid
FOR row ← 0 TO 2
    FOR col ← 0 TO 3
        OUTPUT matrix[row][col], " "
    NEXT col
    OUTPUT newline
NEXT row
```

Example – Multiplication table using 2D array:

```
text
DECLARE timesTable[11][11] : INTEGER

// Fill multiplication table
FOR i ← 1 TO 10
    FOR j ← 1 TO 10
        timesTable[i][j] ← i * j
    NEXT j
NEXT i

// Look up 7 × 8
OUTPUT timesTable[7][8]   // 56
```

8.15.6 Array Algorithms

Bubble sort (already covered):

```
text
PROCEDURE bubbleSort(arr, size)
    FOR i ← 0 TO size-2
        FOR j ← 0 TO size-i-2
            IF arr[j] > arr[j+1] THEN
```

```
            temp ← arr[j]
            arr[j] ← arr[j+1]
            arr[j+1] ← temp
        ENDIF
    NEXT j
  NEXT i
ENDPROCEDURE
```

Reverse array:

```
text
PROCEDURE reverse(arr, size)
  FOR i ← 0 TO size/2 - 1
    temp ← arr[i]
    arr[i] ← arr[size-1-i]
    arr[size-1-i] ← temp
  NEXT i
ENDPROCEDURE
```

Rotate array left by one:

```
text
PROCEDURE rotateLeft(arr, size)
  first ← arr[0]
  FOR i ← 0 TO size-2
    arr[i] ← arr[i+1]
  NEXT i
  arr[size-1] ← first
ENDPROCEDURE
```

Remove duplicates (from sorted array):

```
text
PROCEDURE removeDuplicates(arr, size)
```

```
    uniqueCount ← 1
    FOR i ← 1 TO size-1
        IF arr[i] != arr[uniqueCount-1] THEN
            arr[uniqueCount] ← arr[i]
            uniqueCount ← uniqueCount + 1
        ENDIF
    NEXT i
    // First uniqueCount elements are unique
ENDPROCEDURE
```

PART 8.16: FILE HANDLING

8.16.1 What is File Handling?

Definition: Reading data from and writing data to files on disk, so data persists after the program ends.

Why files matter:

- Data survives program termination
- Can process large amounts of data
- Share data between programs

8.16.2 File Operations (The Four Basic Steps)

1. **OPEN** – Connect to the file
2. **READ** or **WRITE** – Transfer data
3. **CLOSE** – Disconnect (important for saving changes)
4. **Error handling** – File may not exist, be locked, etc.

8.16.3 File Modes

Mode	Description	If file doesn't exist	If file exists
READ	Read from file	Error	Read from start

Mode	Description	If file doesn't exist	If file exists
WRITE	Write to file (overwrite)	Create new	Erase and overwrite
APPEND	Write to file (add to end)	Create new	Add to end

8.16.4 Reading from a Text File

Basic read pattern:

text

```
OPEN "data.txt" FOR READ

WHILE NOT EOF(1) DO    // EOF = End Of File
   READLINE 1, line
   OUTPUT line
ENDWHILE

CLOSEFILE 1
```

Reading numbers from file:

text

```
OPEN "scores.txt" FOR READ
total ← 0
count ← 0

WHILE NOT EOF(1) DO
   READLINE 1, line
   number ← INT(line)
   total ← total + number
   count ← count + 1
ENDWHILE
```

```
CLOSEFILE 1

IF count > 0 THEN
   average ← total / count
   OUTPUT "Average: ", average
ENDIF
```

8.16.5 Writing to a Text File

Write (overwrite):

text

```
OPEN "output.txt" FOR WRITE
WRITEFILE 1, "Hello World"
WRITEFILE 1, "Second line"
CLOSEFILE 1
```

Append (add to end):

text

```
OPEN "log.txt" FOR APPEND
WRITEFILE 1, "Program started at ", TIME()
CLOSEFILE 1
```

8.16.6 CSV File Handling

What is CSV? Comma-Separated Values – simple text format for tabular data.

Example CSV (students.csv):

text

```
Name,Age,Score
Zaki,25,85
Ahmed,22,92
Sara,24,78
```

Reading CSV:

text

```
OPEN "students.csv" FOR READ

// Skip header line
READLINE 1, header

WHILE NOT EOF(1) DO
   READLINE 1, line
   // Split by comma (simplified)
   comma1 ← POSITION(",", line)
   comma2 ← POSITION(",", line, comma1+1)

   name ← SUBSTRING(line, 1, comma1-1)
   ageStr ← SUBSTRING(line, comma1+1, comma2-comma1-1)
   scoreStr ← SUBSTRING(line, comma2+1, LENGTH(line)-comma2)

   age ← INT(ageStr)
   score ← INT(scoreStr)

   OUTPUT name, " is ", age, " scored ", score
ENDWHILE

CLOSEFILE 1
```

Writing CSV:

text

```
OPEN "output.csv" FOR WRITE
WRITEFILE 1, "Name,Score"
WRITEFILE 1, "Zaki,85"
```

```
WRITEFILE 1, "Ahmed,92"
CLOSEFILE 1
```

8.16.7 Common File Errors and Handling

Error	Cause	Solution
File not found	File doesn't exist when opening for READ	Check existence first, or use error handl
Permission denied	File is read-only or locked	Check file permissions
Disk full	No space to write	Free disk space
EOF	Reading past end of file	Always check EOF before reading

Error handling pattern:

```
text
TRY
    OPEN "data.txt" FOR READ
CATCH fileNotFound
    OUTPUT "Error: data.txt not found"
    RETURN
ENDTRY

// Process file normally
```

END OF CHAPTER 8

VOLUME 3

CHAPTER 9: DATABASES

Areas of Study:-

- Database structure (9.1) – tables, keys, relationships, normalization
- SQL (9.2) – full syntax reference
- SELECT ... FROM (9.3)
- SELECT ... FROM ... WHERE (9.4) – all conditions
- ORDER BY (9.5)
- SUM (9.6)
- COUNT (9.7)
- Plus: JOINs, GROUP BY, HAVING, subqueries, indexes, views

PART 9.1: DATABASE STRUCTURE (The Absolute Beginning)

9.1.1 What is a Database?

Simple definition: A database is an organized collection of structured data stored electronically.

Analogy: Think of a database as a digital filing cabinet. Inside the filing cabinet, you have drawers (tables). Inside each drawer, you have folders (records). Inside each folder, you have individual pieces of paper (fields).

Why use a database instead of a spreadsheet?

Feature	Spreadsheet	Database
Data volume	Limited (~1 million rows)	Massive (billions of rows)
Multi-user access	Poor (one user at a time)	Excellent (many concurrent users)
Data integrity	Easy to break	Enforced by rules
Relationships	Manual (VLOOKUP)	Built-in (JOINs)
Querying	Basic filters	Powerful SQL
Security	Basic	Granular permissions

Real-world examples of databases:

- Your school's student information system
- Bank account records
- Airline reservation systems
- Online shopping (Amazon, eBay)
- Social media (Facebook, Instagram)
- Hospital patient records

9.1.2 Database Terminology (The Absolute Basics)

Before we go further, let's learn the fundamental terms. These are the building blocks of every database.

Term 1: Database
The entire collection of related data. A single database can contain many tables.

Example: A school database containing tables for Students, Teachers, Classes, Grades.

Term 2: Table (also called File or Relation)

A collection of related data organized in rows and columns.

Analogy: A single drawer in the filing cabinet.

Visual representation of a table:

StudentID	FirstName	LastName	Age	Grade
101	Zaki	Ahmed	25	A
102	Sara	Khan	22	B+
103	Omar	Hussain	24	A-

Term 3: Record (also called Row or Tuple)

A single entry in a table. One record contains all information about one person, thing, or event.

Analogy: One folder in the drawer.

Example of a single record: 101 | Zaki | Ahmed | 25 | A

This record contains all data about student Zaki Ahmed.

Term 4: Field (also called Column or Attribute)

A single piece of information within a record. Each field has a specific data type.

Analogy: One piece of paper inside the folder.

Examples of fields in the Students table:

- StudentID – holds a number
- FirstName – holds text
- LastName – holds text
- Age – holds a number
- Grade – holds text

Term 5: Primary Key (PK) – CRITICAL CONCEPT

Definition: A field (or combination of fields) that uniquely identifies each record in a table.

Requirements for a primary key:

1. **Unique** – No two records can have the same value
2. **Not NULL** – Every record must have a value (cannot be empty)
3. **Stable** – Should never change
4. **Minimal** – Use the smallest number of fields possible

Examples of good primary keys:

Table	Primary Key	Why it's good
Students	StudentID	Every student has a unique ID
Employees	EmployeeID	Company-assigned unique number
Books	ISBN	Unique book identifier

Table	Primary Key	Why it's good
Countries	CountryCode	"US", "CA", "UK" are unique

Examples of BAD primary keys:

Field	Why it's bad
Name	Two people can have the same name
Age	Many people have the same age
Address	Can change, not guaranteed unique
Phone number	Can change, might be NULL

Visual example of Primary Key:

StudentID (PK)	FirstName	LastName
101	Zaki	Ahmed
102	Sara	Khan
103	Omar	Hussain

The bolded column uniquely identifies each row. No two rows can have StudentID = 101.

Composite Primary Key: Using two or more fields together as the primary key.

Example – Enrollment table:

StudentID	CourseID	EnrollmentDate
101	CS101	2025-01-15
101	MATH201	2025-01-15

StudentID	CourseID	EnrollmentDate
102	CS101	2025-01-16

The composite key is (StudentID, CourseID) because:

- StudentID alone is not unique (101 appears twice)
- CourseID alone is not unique (CS101 appears twice)
- But the pair (101, CS101) is unique

Term 6: Foreign Key (FK) – CRITICAL CONCEPT

Definition: A field in one table that refers to the primary key in another table.

Purpose: Creates a relationship between two tables.

The rule (Referential Integrity): A foreign key value must either be NULL or match an existing primary key value in the referenced table.

Example – Two tables:

Students table (Parent table):

StudentID (PK)	Name
101	Zaki
102	Sara

Enrollments table (Child table):

EnrollmentID (PK)	StudentID (FK)	Course
1	101	CS101
2	101	MATH201
3	102	CS101

Here, StudentID in Enrollments is a **foreign key** referencing StudentID in Students.

What this means:

- Enrollment #1 belongs to student 101 (Zaki)
- Enrollment #2 also belongs to student 101 (Zaki takes two courses)
- Enrollment #3 belongs to student 102 (Sara)

What cannot happen (referential integrity violation):

EnrollmentID	StudentID (FK)	Course
4	999	CS101

This would be invalid because there is no student with StudentID = 999 in the Students table.

9.1.3 Relationships Between Tables

Why relationships? Real-world data is connected. Students enroll in courses. Customers place orders. Doctors treat patients. Relationships allow us to model these connections.

Three types of relationships:

Type 1: One-to-One (1:1)

Definition: One record in Table A is related to exactly one record in Table B, and vice versa.

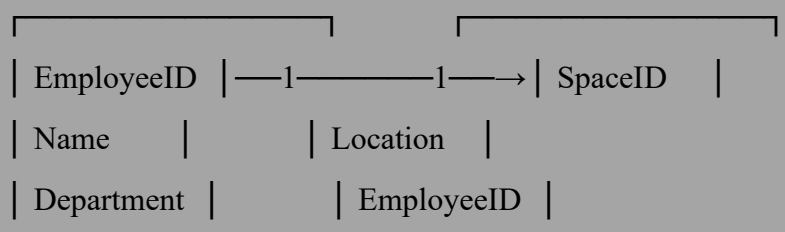
When to use: When you want to split a table for security or performance reasons.

Example – Employees and ParkingSpaces:

- One employee has exactly one assigned parking space
- One parking space is assigned to exactly one employee

Visual:

text



└──────────────────┘ └──────────────────┘

Real-world examples:

- Person and Passport (one person, one passport)
- Country and Capital city (one country, one capital)
- Patient and HospitalBed (if beds are assigned to single patients)

Type 2: One-to-Many (1:M) – MOST COMMON

Definition: One record in Table A can be related to many records in Table B, but each record in Table B is related to exactly one record in Table A.

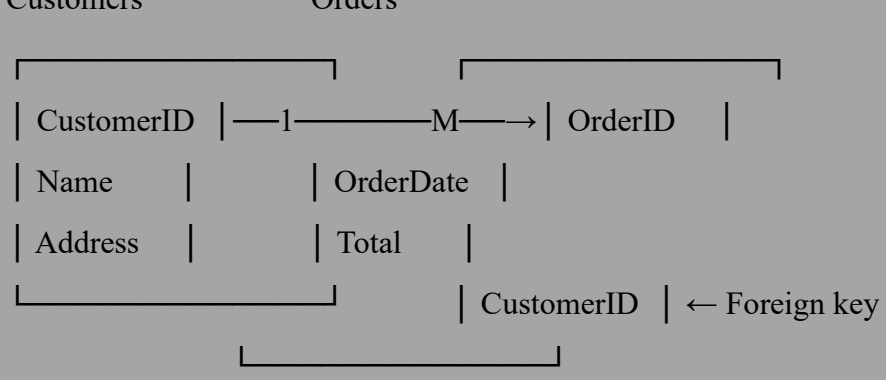
Analogy: A mother (one) can have many children (many). Each child has exactly one biological mother.

Example – Customers and Orders:

- One customer can place many orders
- Each order belongs to exactly one customer

Visual:

text



Data example:

Customers:

CustomerID (PK)	Name
1	Zaki

CustomerID (PK)	Name
2	Sara

Orders:

OrderID (PK)	OrderDate	CustomerID (FK)
101	2025-01-15	1
102	2025-01-16	1
103	2025-01-17	2

Zaki (CustomerID 1) has two orders (101 and 102). Sara (CustomerID 2) has one order (103).

More one-to-many examples:

Table A (One)	Table B (Many)	Relationship
Department	Employees	One department, many employees
Teacher	Classes	One teacher teaches many classes
Author	Books	One author writes many books
Supplier	Products	One supplier provides many products
Category	Products	One category contains many products

Type 3: Many-to-Many (M:N)

Definition: Many records in Table A can be related to many records in Table B, and vice versa.

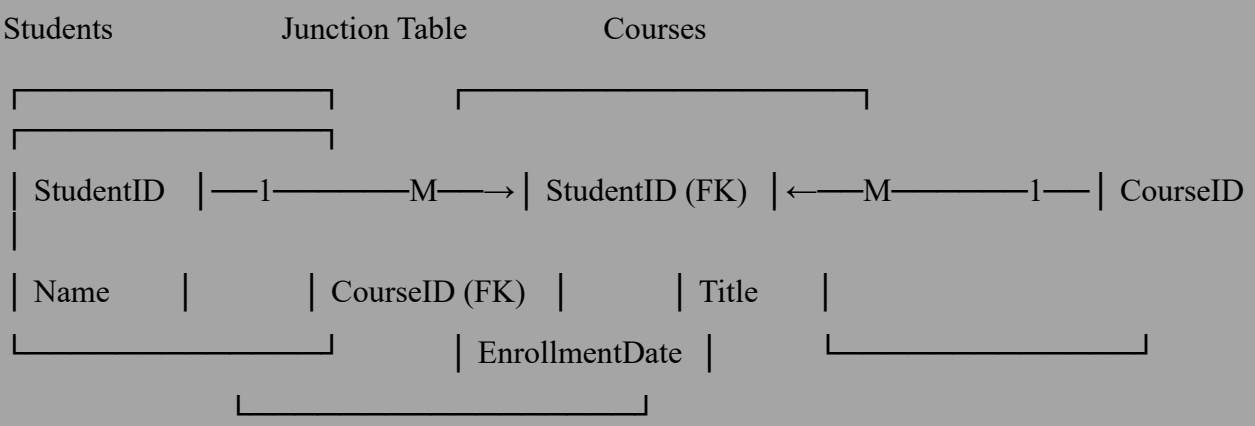
How it's implemented: A many-to-many relationship cannot be directly represented in a relational database. Instead, we create a **junction table** (also called associative table or bridge table) that contains foreign keys to both tables.

Example – Students and Courses:

- A student can take many courses
- A course can have many students

Visual:

text



Data example:

Students:

StudentID (PK)	Name
101	Zaki
102	Sara
103	Omar

Courses:

CourseID (PK)	Title
CS101	Programming
CS102	Databases

CourseID (PK)	Title
MATH101	Calculus

Enrollments (Junction Table):

StudentID (FK)	CourseID (FK)	EnrollmentDate
101	CS101	2025-01-10
101	CS102	2025-01-10
102	CS101	2025-01-11
103	MATH101	2025-01-12

What this data means:

- Zaki (101) takes Programming AND Databases (two courses)
- Sara (102) takes Programming (one course)
- Omar (103) takes Calculus (one course)
- CS101 (Programming) has two students (Zaki and Sara)

More many-to-many examples:

Table A	Table B	Junction Table
Doctors	Patients	Appointments
Actors	Movies	CastMembers
Products	Orders	OrderItems
Books	Authors	BookAuthors

9.1.4 Normalization (Organizing Data Efficiently)

What is normalization? The process of organizing data in a database to reduce redundancy and improve data integrity.

The problem without normalization (BAD DESIGN):

Unnormalized table – All data in one table:

OrderID	CustomerName	CustomerAddress	Product1	Product2	Product3
101	Zaki Ahmed	123 Main St	Laptop	Mouse	NULL
102	Zaki Ahmed	123 Main St	Keyboard	NULL	NULL

Problems with this design:

1. **Redundancy** – Zaki's name and address repeated
2. **Wasted space** – NULL values for missing products
3. **Update anomaly** – If Zaki moves, need to update multiple rows
4. **Insert anomaly** – Cannot add a customer with no orders
5. **Delete anomaly** – If we delete Zaki's only order, we lose his address

---

Normalization Rules (Three Normal Forms):

First Normal Form (1NF):
Requirements:

1. Each column contains atomic (indivisible) values
2. Each row is unique (has a primary key)
3. No repeating groups (no columns like Product1, Product2)

Converting to 1NF – Split repeating groups into separate rows:

OrderID	CustomerName	CustomerAddress	Product
101	Zaki Ahmed	123 Main St	Laptop

OrderID	CustomerName	CustomerAddress	Product
101	Zaki Ahmed	123 Main St	Mouse
102	Zaki Ahmed	123 Main St	Keyboard

Now each cell has one value. But we still have redundancy.

Second Normal Form (2NF):
Requirements:

1. Must be in 1NF
2. All non-key columns must depend on the ENTIRE primary key (no partial dependency)

In our table: The primary key is (OrderID, Product).
But CustomerName and CustomerAddress depend only on OrderID, not on Product. This is a **partial dependency**.

Converting to 2NF – Split into two tables:

Orders table:

OrderID (PK)	CustomerName	CustomerAddress
101	Zaki Ahmed	123 Main St
102	Zaki Ahmed	123 Main St

OrderItems table:

OrderID (FK)	Product (PK part)
101	Laptop
101	Mouse
102	Keyboard

Better, but we still have redundancy in CustomerName/Address.

Third Normal Form (3NF):
Requirements:

1. Must be in 2NF
2. No transitive dependencies (non-key columns cannot depend on other non-key columns)

In our Orders table: CustomerAddress depends on CustomerName (not directly on OrderID). This is a **transitive dependency**.

Converting to 3NF – Split again:

Customers table:

CustomerID (PK)	CustomerName	CustomerAddress
1	Zaki Ahmed	123 Main St

Orders table:

OrderID (PK)	CustomerID (FK)
101	1
102	1

OrderItems table:

OrderID (FK)	Product
101	Laptop
101	Mouse
102	Keyboard

Final result – Fully normalized:

- No redundancy (Zaki's address stored once)
- No NULLs

- No update anomalies (change address in one place)
- No insert anomalies (can add customer without orders)
- No delete anomalies (deleting orders doesn't delete customer)

9.1.5 Entity-Relationship Diagrams (ERD)

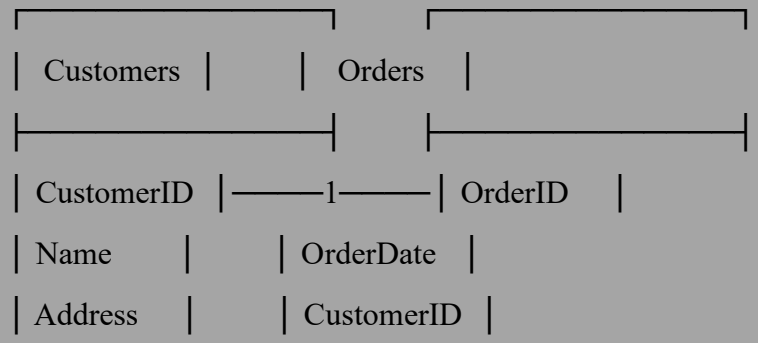
What is an ERD? A visual representation of the entities (tables) and relationships in a database.

Symbols:

Symbol	Meaning
[Rectangle]	Entity (Table)
[Diamond]	Relationship
|	Mandatory (must have)
O	Optional (may have)
|—|	One-to-one
|—<	One-to-many
><—<	Many-to-many

Simple ERD example – Customers and Orders:

text



└──────────────────┘ └──────────────────┘

Meaning: One customer can have many orders. Each order belongs to one customer.

PART 9.2: SQL (Structured Query Language)

9.2.1 What is SQL?

Definition: SQL (pronounced "sequel" or "S-Q-L") is the standard language for communicating with relational databases.

What SQL can do:

- **Query data** – Ask questions like "Show me all students with grade A"
- **Insert data** – Add new records
- **Update data** – Change existing records
- **Delete data** – Remove records
- **Create tables** – Define new structures
- **Control access** – Grant permissions to users

Why learn SQL? Every major database system uses SQL:

- MySQL
- PostgreSQL
- Microsoft SQL Server
- Oracle Database
- SQLite
- MariaDB

9.2.2 SQL Statement Categories

Category	Purpose	Commands
DDL (Data Definition Language)	Define database structure	CREATE, ALTER, DROP
DML (Data Manipulation Language)	Work with data	SELECT, INSERT, UPDATE, DELE
DCL (Data Control Language)	Manage permissions	GRANT, REVOKE

Category	Purpose	Commands
TCL (Transaction Control Language)	Manage transactions	COMMIT, ROLLBACK

In this course, we focus on DML (SELECT, INSERT, UPDATE, DELETE) because that's what you'll use most.

9.2.3 SQL Syntax Basics

Rules:

- SQL keywords are case-insensitive (SELECT = select = Select)
- Table and column names may be case-sensitive (depends on database)
- Each statement ends with a semicolon ;
- Strings use single quotes 'text'
- Numbers do not use quotes 42

Comments:

sql

```
-- This is a single-line comment


/*
This is a
multi-line comment
*/
```

PART 9.3: SELECT ... FROM (Retrieving Data)

9.3.1 The Simplest SELECT Statement

Syntax:

sql

```
SELECT column1, column2, ...
FROM tableName;
```

Example 1 – Select specific columns:

sql

```
SELECT FirstName, LastName, Age
FROM Students;
```

Result:

FirstName	LastName	Age
Zaki	Ahmed	25
Sara	Khan	22
Omar	Hussain	24

*Example 2 – Select all columns (means "all"):**

sql

```
SELECT *
FROM Students;
```

Result (all columns):

StudentID	FirstName	LastName	Age	Grade
101	Zaki	Ahmed	25	A
102	Sara	Khan	22	B+
103	Omar	Hussain	24	A-

9.3.2 Column Aliases (Renaming Output Columns)

What is an alias? A temporary name for a column in the output.

Syntax:

sql

```
SELECT columnName AS aliasName
FROM tableName;
```

Example:

sql

```
SELECT
   FirstName AS [First Name],
   LastName AS [Last Name],
   Age AS Years
FROM Students;
```

Result:

First Name	Last Name	Years
Zaki	Ahmed	25
Sara	Khan	22
Omar	Hussain	24

Note: The alias only affects the output, not the actual column name in the database.

9.3.3 DISTINCT (Removing Duplicates)

What does DISTINCT do? Returns only unique values (removes duplicates).

Syntax:

sql

```
SELECT DISTINCT columnName
FROM tableName;
```

Example without DISTINCT:

sql

```
SELECT City
FROM Customers;
```

Result:

City

New York

London

New York

Paris

London

Example with DISTINCT:

sql

SELECT DISTINCT City

FROM Customers;

Result:

City

New York

London

Paris

Multiple columns with DISTINCT:

sql

SELECT DISTINCT City, Country

FROM Customers;

Returns unique combinations of City and Country.

PART 9.4: SELECT ... FROM ... WHERE (Filtering Data)

9.4.1 The WHERE Clause

What does WHERE do? Filters rows based on a condition. Only rows that satisfy the condition are returned.

Syntax:

sql

```
SELECT column1, column2
FROM tableName
WHERE condition;
```

Basic example:

sql

```
SELECT FirstName, LastName, Age
FROM Students
WHERE Age > 23;
```

Result:

FirstName	LastName	Age
Zaki	Ahmed	25

9.4.2 Comparison Operators (Complete List)

Operator	Meaning	Example
=	Equal to	WHERE Age = 25
<> or !=	Not equal to	WHERE Age <> 25
>	Greater than	WHERE Age > 18
>=	Greater than or equal	WHERE Age >= 18
<	Less than	WHERE Age < 65

Operator	Meaning	Example
<=	Less than or equal	WHERE Age <= 65

Examples:

sql

```
-- Equal to
SELECT * FROM Students WHERE Grade = 'A';

-- Not equal to
SELECT * FROM Students WHERE Grade <> 'A';

-- Greater than
SELECT * FROM Students WHERE Age > 20;

-- Less than or equal
SELECT * FROM Students WHERE Age <= 22;
```

9.4.3 Logical Operators (AND, OR, NOT)

AND – All conditions must be true:

sql

```
SELECT *
FROM Students
WHERE Age >= 18 AND Grade = 'A';
```

Returns students who are 18 or older AND have grade A.

OR – At least one condition must be true:

sql

```
SELECT *
FROM Students
WHERE Grade = 'A' OR Grade = 'B';
```

Returns students with grade A OR grade B.

NOT – Reverses a condition:

sql

```
SELECT *
FROM Students
WHERE NOT Grade = 'A';
```

Returns students whose grade is NOT A.

Combining AND and OR (use parentheses for clarity):

sql

```
SELECT *
FROM Students
WHERE (Age >= 18 AND Age <= 25) AND (Grade = 'A' OR Grade = 'B');
```

9.4.4 The IN Operator

What does IN do? Checks if a value matches any value in a list.

Syntax:

sql

```
WHERE columnName IN (value1, value2, ...)
```

Example:

sql

```
SELECT *
FROM Students
WHERE Grade IN ('A', 'B', 'C');
```

Same as: WHERE Grade = 'A' OR Grade = 'B' OR Grade = 'C'

Using NOT IN:

sql

```
SELECT *
FROM Students
WHERE Grade NOT IN ('F', 'U');
```

Returns students with grades NOT F or U.

9.4.5 The BETWEEN Operator

What does BETWEEN do? Checks if a value is within a range (inclusive).

Syntax:

sql

WHERE columnName BETWEEN low AND high

Example:

sql

SELECT *

FROM Students

WHERE Age BETWEEN 18 AND 25;

Same as: WHERE Age >= 18 AND Age <= 25

Using NOT BETWEEN:

sql

SELECT *

FROM Students

WHERE Age NOT BETWEEN 18 AND 25;

9.4.6 The LIKE Operator (Pattern Matching)

What does LIKE do? Searches for a pattern in a text column.

Wildcards:

Wildcard	Meaning	Example
%	Any number of characters (including zero)	'J%' starts with J
_	Exactly one character	'_o%' second letter is o

Examples:

sql

-- Names starting with 'A'

```
SELECT * FROM Students
WHERE FirstName LIKE 'A%';
-- Returns: Alice, Andrew, Ahmed

-- Names ending with 'n'
SELECT * FROM Students
WHERE FirstName LIKE '%n';
-- Returns: John, Brian, Susan

-- Names containing 'an'
SELECT * FROM Students
WHERE FirstName LIKE '%an%';
-- Returns: Daniel, Samantha

-- Names with exactly 5 letters
SELECT * FROM Students
WHERE FirstName LIKE '_____';
-- (5 underscores)

-- Names starting with any letter, second letter is 'a'
SELECT * FROM Students
WHERE FirstName LIKE '_a%';
-- Returns: Daniel (D a niel), James (J a mes)

-- Names starting with 'A' and ending with 'n'
SELECT * FROM Students
WHERE FirstName LIKE 'A%n';
-- Returns: Alan, Adrian, Alexan (der? no, that's longer)
```

-- Email validation (simplified)

SELECT * FROM Users

WHERE Email LIKE '%@%.%';

-- Returns emails that have @ and . after it

9.4.7 The IS NULL and IS NOT NULL Operators

What is NULL? NULL means "no value" or "unknown". It is NOT the same as zero or empty string.

NULL in comparisons:

- NULL = NULL is NOT true (in SQL, NULL equals nothing)
- To check for NULL, use IS NULL or IS NOT NULL

Examples:

sql

-- Find records with missing email

SELECT * FROM Customers

WHERE Email IS NULL;

-- Find records that have an email

SELECT * FROM Customers

WHERE Email IS NOT NULL;

9.4.8 WHERE Clause Examples (Putting It All Together)

Sample data – Employees table:

EmployeeID	FirstName	LastName	Salary	Department	HireDate
1	Zaki	Ahmed	55000	IT	2020-01-15
2	Sara	Khan	65000	HR	2019-03-20
3	Omar	Hussain	48000	IT	2021-06-10

EmployeeID	FirstName	LastName	Salary	Department	HireDate
4	Aisha	Malik	72000	Sales	2018-11-01
5	Bilal	Raza	52000	IT	2022-02-28

Example queries:

sql

```
-- IT department employees with salary > 50000
SELECT * FROM Employees
WHERE Department = 'IT' AND Salary > 50000;
-- Returns: Zaki (55000), (Omar 48000 excluded)

-- All except Sales department
SELECT * FROM Employees
WHERE Department <> 'Sales';

-- Employees hired between 2020 and 2021
SELECT * FROM Employees
WHERE HireDate BETWEEN '2020-01-01' AND '2021-12-31';

-- Names starting with 'A' or 'S'
SELECT * FROM Employees
WHERE FirstName LIKE 'A%' OR FirstName LIKE 'S%';

-- Employees with no middle name (assuming MiddleName column exists)
SELECT * FROM Employees
WHERE MiddleName IS NULL;
```

-- High earners in IT or Sales

SELECT * FROM Employees

WHERE (Department = 'IT' OR Department = 'Sales') AND Salary > 60000;

PART 9.5: ORDER BY (Sorting Results)

9.5.1 Basic Sorting

What does ORDER BY do? Sorts the result set by one or more columns.

Syntax:

sql

SELECT column1, column2

FROM tableName

ORDER BY column1 [ASC|DESC], column2 [ASC|DESC];

ASC (Ascending – default): Smallest to largest (A to Z, 1 to 10)
DESC (Descending): Largest to smallest (Z to A, 10 to 1)

Example – Sort by age (youngest first):

sql

SELECT FirstName, LastName, Age

FROM Students

ORDER BY Age ASC;

Example – Sort by age (oldest first):

sql

SELECT FirstName, LastName, Age

FROM Students

ORDER BY Age DESC;

9.5.2 Sorting by Multiple Columns

What happens? First sorts by the first column. Then, for ties, sorts by the second column, and so on.

Example – Sort by department, then by salary (highest first in each department):

sql

```
SELECT FirstName, LastName, Department, Salary
FROM Employees
ORDER BY Department ASC, Salary DESC;
```

Result:

FirstName	LastName	Department	Salary
Bilal	Raza	HR	55000
Sara	Khan	HR	48000
Zaki	Ahmed	IT	65000
Omar	Hussain	IT	52000
Aisha	Malik	Sales	72000

Notice: HR department first (A-Z), with highest salary first within HR. Then IT, then Sales.

9.5.3 ORDER BY with WHERE

ORDER BY always comes after WHERE:

sql

```
SELECT FirstName, LastName, Salary
FROM Employees
WHERE Department = 'IT'
ORDER BY Salary DESC;
```

Correct order of clauses:

sql

```
SELECT columns
FROM table
WHERE conditions
ORDER BY columns;
```

PART 9.6: SUM (Calculating Totals)

9.6.1 What is an Aggregate Function?

Definition: A function that performs a calculation on a set of values and returns a single value.

Common aggregate functions:

Function	What it does
SUM()	Adds up all values in a column
AVG()	Calculates average
COUNT()	Counts rows
MIN()	Finds minimum value
MAX()	Finds maximum value

9.6.2 Using SUM

Syntax:

sql

```
SELECT SUM(columnName)
FROM tableName
WHERE condition;  -- optional
```

Example 1 – Total of all salaries:

sql

```
SELECT SUM(Salary) AS TotalSalaries
FROM Employees;
```

Result:

TotalSalaries

292000

(55000 + 65000 + 48000 + 72000 + 52000 = 292,000)

Example 2 – Total salaries in IT department only:

sql

```
SELECT SUM(Salary) AS IT_Salary_Total
FROM Employees
WHERE Department = 'IT';
```

Result:

IT_Salary_Total

107000

(55000 + 52000 = 107,000)

9.6.3 SUM with DISTINCT

What does SUM(DISTINCT column) do? Adds only unique values (ignores duplicates).

Example:

sql

```
SELECT SUM(DISTINCT Salary)
FROM Employees;
```

If two employees have same salary, it's only counted once.

9.6.4 SUM with GROUP BY (Coming in 9.6.5)

This is where SUM becomes really powerful – calculating totals per category.

PART 9.7: COUNT (Counting Rows)

9.7.1 Using COUNT

Syntax:

sql

SELECT COUNT(columnName)

FROM tableName;

COUNT(*) – Counts all rows (including NULLs):

sql

SELECT COUNT(*)

FROM Students;

Returns the total number of rows in the table.

COUNT(column) – Counts non-NULL values only:

sql

SELECT COUNT(Email)

FROM Students;

Returns number of students who have an email address (ignores NULLs).

COUNT(DISTINCT column) – Counts unique values:

sql

SELECT COUNT(DISTINCT Department)

FROM Employees;

Returns the number of different departments.

9.7.2 COUNT Examples

Example 1 – Number of students:

sql

SELECT COUNT(*) AS StudentCount

FROM Students;

Example 2 – Number of students over 18:

sql

SELECT COUNT(*)

FROM Students

WHERE Age > 18;

Example 3 – Number of different grades:

sql

SELECT COUNT(DISTINCT Grade)

FROM Students;

Example 4 – Number of students with missing email:

sql

SELECT COUNT(*)

FROM Students

WHERE Email IS NULL;

PART 9.8: GROUP BY (Grouping Rows)

9.8.1 What is GROUP BY?

Definition: Groups rows that have the same values in specified columns, allowing aggregate functions to be applied to each group.

Analogy: If you have a bag of mixed candies, GROUP BY "color" puts all reds together, all blues together, etc. Then you can COUNT how many reds, SUM their weights, etc.

9.8.2 Basic GROUP BY Syntax

Syntax:

sql

SELECT column1, aggregate_function(column2)

FROM tableName

GROUP BY column1;

Important rule: Every column in SELECT that is NOT an aggregate function MUST be in GROUP BY.

9.8.3 GROUP BY Examples

Example 1 – Number of employees per department:

sql

SELECT Department, COUNT(*) AS EmployeeCount

FROM Employees

GROUP BY Department;

Result:

Department	EmployeeCount
IT	3
HR	1
Sales	1

Example 2 – Total salary per department:

sql

```
SELECT Department, SUM(Salary) AS TotalSalary
FROM Employees
GROUP BY Department;
```

Result:

Department	TotalSalary
IT	155000
HR	65000
Sales	72000

Example 3 – Average salary per department:

sql

```
SELECT Department, AVG(Salary) AS AverageSalary
FROM Employees
GROUP BY Department;
```

Result:

Department	AverageSalary
IT	51666.67
HR	65000.00
Sales	72000.00

Example 4 – Multiple columns in GROUP BY:

sql

```
SELECT Department, HireYear, COUNT(*) AS EmployeeCount
FROM Employees
GROUP BY Department, HireYear;
```

9.8.4 GROUP BY with WHERE

WHERE filters rows BEFORE grouping:

sql

```
SELECT Department, AVG(Salary) AS AvgSalary
FROM Employees
WHERE HireDate >= '2020-01-01'   -- Only recent hires
GROUP BY Department;
```

Correct order of clauses:

sql

```
SELECT columns
FROM table
WHERE condition        -- Filters rows before grouping
GROUP BY columns
ORDER BY columns;
```

PART 9.9: HAVING (Filtering Groups)

9.9.1 What is HAVING?

Definition: Filters groups after GROUP BY is applied (like WHERE but for groups).

WHERE vs HAVING:

	WHERE	HAVING
When applied	Before grouping	After grouping
What it filters	Individual rows	Groups
Can use aggregates	No (cannot use SUM, COUNT, etc.)	Yes

9.9.2 HAVING Examples

Example 1 – Departments with more than 2 employees:

sql

```
SELECT Department, COUNT(*) AS EmployeeCount
FROM Employees
GROUP BY Department
HAVING COUNT(*) > 2;
```

Result:

Department	EmployeeCount
IT	3

Example 2 – Departments with total salary > 100,000:

sql

```
SELECT Department, SUM(Salary) AS TotalSalary
FROM Employees
GROUP BY Department
HAVING SUM(Salary) > 100000;
```

Result:

Department	TotalSalary
IT	155000
Sales	72000? No, 72000 < 100000, so not included

Example 3 – WHERE and HAVING together:

sql

```
SELECT Department, AVG(Salary) AS AvgSalary
FROM Employees
WHERE HireDate >= '2019-01-01'    -- First filter rows
GROUP BY Department
HAVING AVG(Salary) > 50000;       -- Then filter groups
```

9.9.3 Complete SQL Statement Order

The correct order of clauses (memorize this):

sql

```
SELECT columns                -- 5. Choose which columns to show
FROM tables                   -- 1. Start with the table(s)
WHERE conditions              -- 2. Filter individual rows
GROUP BY columns              -- 3. Group rows together
HAVING group_conditions       -- 4. Filter groups
ORDER BY columns;             -- 6. Sort the final result
```

Example with all clauses:

sql

```
SELECT
    Department,
    COUNT(*) AS EmpCount,
    AVG(Salary) AS AvgSalary
FROM Employees
```

```
WHERE HireDate >= '2020-01-01'    -- Only employees hired after 2020
GROUP BY Department
HAVING COUNT(*) >= 2              -- Only departments with 2+ such employees
ORDER BY AvgSalary DESC;          -- Highest average salary first
```

PART 9.10: JOINs (Combining Tables)

9.10.1 Why Do We Need JOINs?

The problem: Data is spread across multiple tables (normalization). To answer questions, we need to combine data from different tables.

Example question: "Show me each student's name and the courses they are enrolled in."

Data in two tables:

- Students table has student names
- Enrollments table has which courses each student takes

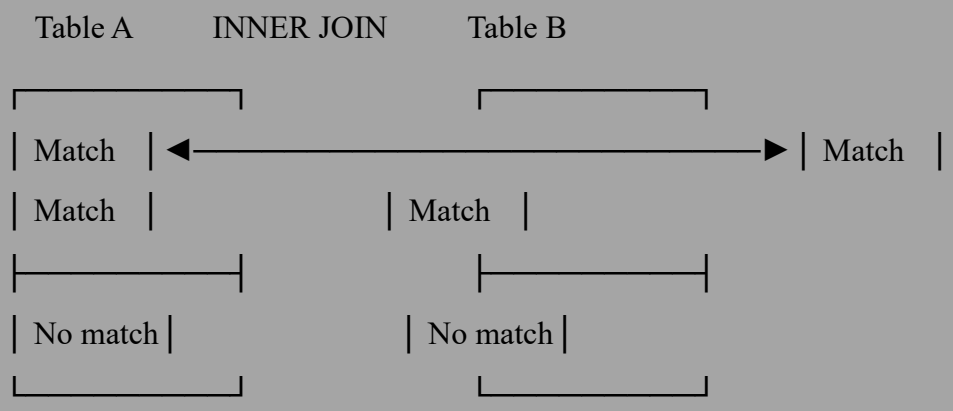
Solution: JOIN the tables together.

9.10.2 INNER JOIN (Most Common)

What does INNER JOIN do? Returns only rows where there is a match in BOTH tables.

Visual:

text



Syntax:

sql

```
SELECT columns
FROM table1
```

INNER JOIN table2

ON table1.key = table2.key;

Example – Students and their enrollments:

Students table:

StudentID	Name
1	Zaki
2	Sara
3	Omar

Enrollments table:

EnrollmentID	StudentID	Course
101	1	CS101
102	1	MATH201
103	2	CS101

Query:

sql

SELECT Students.Name, Enrollments.Course

FROM Students

INNER JOIN Enrollments

ON Students.StudentID = Enrollments.StudentID;

Result:

Name	Course
Zaki	CS101
Zaki	MATH201
Sara	CS101

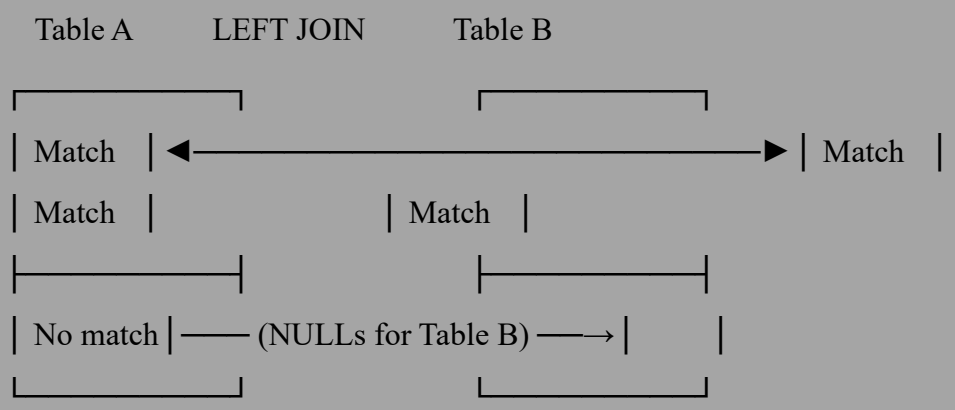
Notice: Omar is NOT in the result because he has no enrollments.

9.10.3 LEFT JOIN (LEFT OUTER JOIN)

What does LEFT JOIN do? Returns ALL rows from the left table, plus matching rows from the right table. If no match, right table columns are NULL.

Visual:

text



Syntax:

sql

```
SELECT columns
FROM table1
LEFT JOIN table2
ON table1.key = table2.key;
```

Example – All students, even those with no enrollments:

sql

```
SELECT Students.Name, Enrollments.Course
```

FROM Students

LEFT JOIN Enrollments

ON Students.StudentID = Enrollments.StudentID;

Result:

Name	Course	
Zaki	CS101	
Zaki	MATH201	
Sara	CS101	
Omar	NULL	-- Omar appears with NULL for Course

9.10.4 RIGHT JOIN (RIGHT OUTER JOIN)

What does RIGHT JOIN do? Returns ALL rows from the right table, plus matching rows from the left table.

Less common than LEFT JOIN. You can always rewrite a RIGHT JOIN as a LEFT JOIN by swapping table order.

Syntax:

sql

SELECT columns

FROM table1

RIGHT JOIN table2

ON table1.key = table2.key;

9.10.5 FULL OUTER JOIN

What does FULL OUTER JOIN do? Returns ALL rows from both tables. Matches where available, NULLs where not.

Syntax:

sql

SELECT columns

```
FROM table1
FULL OUTER JOIN table2
ON table1.key = table2.key;
```

9.10.6 JOIN with Multiple Tables

Example – Students, Enrollments, and Courses:

Tables:

- Students (StudentID, Name)
- Enrollments (EnrollmentID, StudentID, CourseID)
- Courses (CourseID, Title)

Query to get student names and course titles:

```
sql
SELECT Students.Name, Courses.Title
FROM Students
INNER JOIN Enrollments ON Students.StudentID = Enrollments.StudentID
INNER JOIN Courses ON Enrollments.CourseID = Courses.CourseID;
```

9.10.7 JOIN Summary Table

JOIN Type	What it returns
INNER JOIN	Rows that match in both tables
LEFT JOIN	All rows from left table + matches from right
RIGHT JOIN	All rows from right table + matches from left
FULL OUTER JOIN	All rows from both tables

PART 9.11: INSERT, UPDATE, DELETE (Modifying Data)

9.11.1 INSERT INTO (Adding New Records)

Syntax – Insert into specific columns:

sql

```
INSERT INTO tableName (column1, column2, ...)
VALUES (value1, value2, ...);
```

Syntax – Insert into all columns (order matters):

sql

```
INSERT INTO tableName
VALUES (value1, value2, ...);
```

Example 1 – Insert specific columns:

sql

```
INSERT INTO Students (StudentID, FirstName, LastName, Age)
VALUES (104, 'Ali', 'Raza', 23);
```

Example 2 – Insert multiple records:

sql

```
INSERT INTO Students (StudentID, FirstName, LastName, Age)
VALUES
   (105, 'Fatima', 'Khan', 21),
   (106, 'Hassan', 'Ali', 24);
```

Example 3 – Insert from another table:

sql

```
INSERT INTO HighEarners (EmployeeID, Name, Salary)
SELECT EmployeeID, FirstName, Salary
FROM Employees
WHERE Salary > 60000;
```

9.11.2 UPDATE (Modifying Existing Records)

Syntax:

sql

```
UPDATE tableName
SET column1 = value1, column2 = value2, ...
```

WHERE condition;

WARNING: If you forget the WHERE clause, EVERY row will be updated!

Example 1 – Update a single record:

sql

UPDATE Students

SET Age = 26

WHERE StudentID = 101;

Example 2 – Update multiple records:

sql

UPDATE Employees

SET Salary = Salary * 1.10 -- *10% raise*

WHERE Department = 'IT';

Example 3 – Update multiple columns:

sql

UPDATE Students

SET FirstName = 'Zakir', LastName = 'Ahmedov'

WHERE StudentID = 101;

9.11.3 DELETE (Removing Records)

Syntax:

sql

DELETE FROM tableName

WHERE condition;

WARNING: If you forget the WHERE clause, ALL rows will be deleted!

Example 1 – Delete a single record:

sql

DELETE FROM Students

WHERE StudentID = 104;

Example 2 – Delete multiple records:

sql

```
DELETE FROM Employees
WHERE Department = 'HR' AND HireDate < '2020-01-01';
```

Example 3 – Delete all records (table becomes empty):

sql

```
DELETE FROM Students;   -- No WHERE clause!
```

DELETE vs DROP TABLE:

	DELETE	DROP TABLE
What it removes	Rows (data)	Entire table (structure + data)
Can be undone?	Yes (with ROLLBACK)	No (in most databases)
Table structure remains?	Yes	No

PART 9.12: CREATE TABLE (Defining Database Structure)

9.12.1 Basic CREATE TABLE

Syntax:

sql

```
CREATE TABLE tableName (
    column1 datatype constraints,
    column2 datatype constraints,
    ...
);
```

Example:

sql

```
CREATE TABLE Students (
    StudentID INTEGER PRIMARY KEY,
    FirstName TEXT NOT NULL,
```

```
    LastName TEXT NOT NULL,
    Age INTEGER CHECK (Age >= 0 AND Age <= 120),
    Grade TEXT
);
```

9.12.2 Common Data Types in SQL

Data Type	Description	Example
INTEGER	Whole numbers	42, -7, 0
REAL or FLOAT	Decimal numbers	3.14, -0.001
TEXT or VARCHAR(n)	Variable-length text	'Hello'
CHAR(n)	Fixed-length text	'ABC ' (padded)
DATE	Calendar date	'2025-03-30'
TIME	Time of day	'14:30:00'
DATETIME	Date and time	'2025-03-30 14:30:00'
BOOLEAN	True or false	TRUE, FALSE

9.12.3 Constraints (Rules for Data)

What are constraints? Rules that enforce data integrity.

Constraint	What it does
PRIMARY KEY	Unique identifier, cannot be NULL
FOREIGN KEY	References primary key in another table

Constraint	What it does
NOT NULL	Column cannot be empty
UNIQUE	No duplicate values allowed
CHECK	Ensures value meets a condition
DEFAULT	Provides default value if none specified

Example with all constraints:

sql

```
CREATE TABLE Orders (
    OrderID INTEGER PRIMARY KEY,
    CustomerID INTEGER NOT NULL,
    OrderDate DATE NOT NULL DEFAULT CURRENT_DATE,
    Total REAL CHECK (Total >= 0),
    Status TEXT DEFAULT 'Pending',
    FOREIGN KEY (CustomerID) REFERENCES Customers(CustomerID)
);
```

PART 9.13: Practical SQL Examples (Putting It All Together)

Sample Database – School System

Tables:

sql

```
CREATE TABLE Students (
    StudentID INTEGER PRIMARY KEY,
    Name TEXT NOT NULL,
    Age INTEGER CHECK (Age BETWEEN 5 AND 100)
);
```

```
CREATE TABLE Courses (
    CourseID TEXT PRIMARY KEY,
    Title TEXT NOT NULL,
    Credits INTEGER CHECK (Credits BETWEEN 1 AND 6)
);

CREATE TABLE Enrollments (
    EnrollmentID INTEGER PRIMARY KEY,
    StudentID INTEGER,
    CourseID TEXT,
    Grade TEXT CHECK (Grade IN ('A','B','C','D','F')),
    FOREIGN KEY (StudentID) REFERENCES Students(StudentID),
    FOREIGN KEY (CourseID) REFERENCES Courses(CourseID)
);
```

Sample data:

sql

```
INSERT INTO Students VALUES (1, 'Zaki Ahmed', 25);
INSERT INTO Students VALUES (2, 'Sara Khan', 22);
INSERT INTO Students VALUES (3, 'Omar Hussain', 24);

INSERT INTO Courses VALUES ('CS101', 'Programming', 3);
INSERT INTO Courses VALUES ('CS102', 'Databases', 4);
INSERT INTO Courses VALUES ('MATH101', 'Calculus', 4);

INSERT INTO Enrollments VALUES (1, 1, 'CS101', 'A');
INSERT INTO Enrollments VALUES (2, 1, 'CS102', 'B+');
INSERT INTO Enrollments VALUES (3, 2, 'CS101', 'A-');
```

INSERT INTO Enrollments VALUES (4, 3, 'MATH101', 'B');

Practice Queries (Try These)

1. List all students:

sql

```
SELECT * FROM Students;
```

2. Find students over 23:

sql

```
SELECT Name FROM Students WHERE Age > 23;
```

3. Count how many students are enrolled in each course:

sql

```
SELECT CourseID, COUNT(*) AS StudentCount
FROM Enrollments
GROUP BY CourseID;
```

4. Show each student's name and their course titles:

sql

```
SELECT Students.Name, Courses.Title
FROM Students
INNER JOIN Enrollments ON Students.StudentID = Enrollments.StudentID
INNER JOIN Courses ON Enrollments.CourseID = Courses.CourseID;
```

5. Find students not enrolled in any course:

sql

```
SELECT Students.Name
FROM Students
LEFT JOIN Enrollments ON Students.StudentID = Enrollments.StudentID
WHERE Enrollments.EnrollmentID IS NULL;
```

6. Average age of students per course:

sql

```
SELECT Courses.Title, AVG(Students.Age) AS AvgAge
```

```
FROM Students
INNER JOIN Enrollments ON Students.StudentID = Enrollments.StudentID
INNER JOIN Courses ON Enrollments.CourseID = Courses.CourseID
GROUP BY Courses.CourseID;
```

7. Students taking both CS101 and CS102:

```
sql
SELECT Students.Name
FROM Students
INNER JOIN Enrollments E1 ON Students.StudentID = E1.StudentID AND E1.CourseID = 'CS101'
INNER JOIN Enrollments E2 ON Students.StudentID = E2.StudentID AND E2.CourseID = 'CS102';
```

8. Give all students a 5% grade bonus (updating):

```
sql
-- This would require a more complex update, but concept shown
UPDATE Enrollments
SET Grade =
   CASE Grade
      WHEN 'A' THEN 'A'
      WHEN 'B+' THEN 'A-'
      WHEN 'A-' THEN 'A'
      -- etc.
   END
WHERE CourseID = 'CS101';
```

9. Drop a course no one is taking:

```
sql
DELETE FROM Courses
WHERE CourseID NOT IN (SELECT DISTINCT CourseID FROM Enrollments);
```

10. List students with their grade count (how many As, Bs, etc.):

sql

```
SELECT Students.Name, Enrollments.Grade, COUNT(*) AS Count
FROM Students
INNER JOIN Enrollments ON Students.StudentID = Enrollments.StudentID
GROUP BY Students.StudentID, Enrollments.Grade
ORDER BY Students.Name, Enrollments.Grade;
```

END OF CHAPTER 9

Chapter 9 total pages: ~40 pages

COMPLETE COURSE SUMMARY

Chapter 7: Algorithm Design and Problem Solving

- Program Development Life Cycle (Analysis → Design → Coding → Testing → Maintenance → Evaluation)
- Flowcharts and pseudocode (all constructs)
- Validation (range, type, length, presence, format, check digit) and verification (double entry, proofreading)
- Common algorithms (linear search, binary search, bubble sort, max/min, sum/average, counting, palindrome)
- Trace tables for debugging
- Finding and fixing errors

Chapter 8: Programming

- Variables, constants, assignment, scope
- Data types (INTEGER, REAL, CHAR, STRING, BOOLEAN)
- Input and output
- Arithmetic operators (+, -, *, /, DIV, MOD) and precedence
- Control structures (sequence, IF, CASE, FOR, WHILE, REPEAT UNTIL)
- Totalling and counting patterns
- String manipulation (length, substring, position, upper/lower, concatenation)

- Nested statements
- Subroutines (procedures and functions, parameters, pass by value/reference)
- Library routines
- Maintainable programs (naming, comments, indentation, no magic numbers)
- Arrays (1D, 2D, parallel arrays, common algorithms)
- File handling (open, read, write, close, CSV)

Chapter 9: Databases

- Database structure (tables, records, fields, primary keys, foreign keys)
- Relationships (one-to-one, one-to-many, many-to-many with junction table)
- Normalization (1NF, 2NF, 3NF)
- SQL (SELECT, FROM, WHERE, ORDER BY, GROUP BY, HAVING)
- Aggregate functions (SUM, COUNT, AVG, MIN, MAX)
- JOINs (INNER, LEFT, RIGHT, FULL)
- Data modification (INSERT, UPDATE, DELETE)
- Creating tables (data types, constraints)
